THIRSTY'S CODEX

# STANDARDS EDITION

VOLUME IX / 09 OF 20

## CONFIGURATION THROUGH DELIVERY INFRASTRUCTURE

Lifecycle Sections 258-300

NO PROOF, NO CLAIM

CONTINUOUS MASTER FOLIO / SOURCE-PRESERVING EDITION

# Edition Statement

THIRSTY'S CODEX / STANDARDS EDITION / SOURCE-PRESERVING SET

This volume is one part of a continuous 20-volume edition. Canonical lifecycle section numbers remain stable across the set; no volume restarts or renumbers them.

The visual treatment has been rebuilt as a unified standards library: coordinated covers, volume architecture, source dividers, running headers, continuous master folios, local page numbers, and source-identity marks.

The source corpus remains authoritative over the presentation layer. Status terms are preserved exactly. CONTENT-COMPLETE does not silently become source-reconciled, hostile-reviewed, owner-accepted, ratified, or published.

Every registered source witness is identified by path, byte count, and SHA-256 digest. The complete source archive is preserved as an embedded attachment in Volume XX, while the two supplied foundational PDFs are preserved as attachments in Volume I.

**CONTINUITY RULE**

Read the master folio and canonical section sequence as one uninterrupted work. Each succeeding volume begins where the preceding volume ends.

# Standards Edition Set Map

VOLUME I
**Foundations and Governance**
Doctrine, prompts, front matter, shared governance, roles, and output contracts

VOLUME II
**Research, Strategy, and Risk**
Lifecycle Sections 001-018

VOLUME III
**Requirements through Hardware Architecture**
Lifecycle Sections 019-060

VOLUME IV
**Electrical Engineering through User Validation**
Lifecycle Sections 061-106

VOLUME V
**Software Systems through Smart Contracts**
Lifecycle Sections 107-150

VOLUME VI
**Data, Intelligence, and Human Oversight**
Lifecycle Sections 151-188

VOLUME VII
**Cybersecurity through Privacy Engineering**
Lifecycle Sections 189-218

VOLUME VIII
**Law, Assurance, and Validation**
Lifecycle Sections 219-257

VOLUME IX
**Configuration through Delivery Infrastructure**
Lifecycle Sections 258-300

VOLUME X
**Production Operations and Command**
Lifecycle Sections 301-332

VOLUME XI
**Support, Enterprise, and Roadmaps**
Lifecycle Sections 333-373

VOLUME XII
**Maintenance, Decommissioning, and Retirement**
Lifecycle Sections 374-406

VOLUME XIII
**Reference Registers and Working Documentation**
Indexes, cross-references, source registers, checkpoints, and operational guides

VOLUME XIV
**Standards Source Library**
Preserved standards, command editions, blueprints, and source artifacts

VOLUME XV
**Canonical Authority and Assurance**
Authority, constitutions, policies, schemas, provenance, evaluation, and verifica...

VOLUME XVI
**Applied Corpus and V3Q**
Agent-output corpus, V3Q implementation, fixtures, tests, and governed examples

VOLUME XVII
**Training Views and Indexes**
Continuous-pretraining, evaluation, retrieval, and training views with canonical ...

VOLUME XVIII
**Agents and Owner Command Standards**
Agents 101 and Owner's Cockpit source-standard facsimiles

VOLUME XIX
**Static Analysis and Tooling Evidence**
Codacy configuration, rulesets, analysis logs, and repository tooling

# Volume Contents



337 Section 258 - Configuration Management 6
338 Section 259 - Source Control Management 10
339 Section 260 - Dependency Management 14
340 Section 261 - Package Management 18
341 Section 262 - Artifact Management 22
342 Section 263 - Environment Management 26
343 Section 264 - Build Engineering 30
344 Section 265 - Build Automation 34
345 Section 266 - Continuous Integration 38
346 Section 267 - Continuous Delivery 42
347 Section 268 - Continuous Deployment 46
348 Section 269 - Devops 50
349 Section 270 - Devsecops 54
350 Section 271 - Gitops 58
351 Section 272 - Platform Engineering 62
352 Section 273 - Developer Experience 66
353 Section 274 - Tooling Engineering 70
354 Section 275 - Infrastructure Engineering 74
355 Section 276 - Infrastructure As Code 78
356 Section 277 - Cloud Engineering 82
357 Section 278 - Edge Computing Engineering 86
358 Section 279 - Container Engineering 90
359 Section 280 - Container Orchestration 94
360 Section 281 - Virtualization Engineering 98
361 Section 282 - Server Engineering 102
362 Section 283 - Systems Administration 106
363 Section 284 - Network Engineering 110
364 Section 285 - Database Administration 114
365 Section 286 - Storage Administration 118
366 Section 287 - Release Engineering 122
367 Section 288 - Release Management 126

# Volume Contents



368 Section 289 - Change Management 130
369 Section 290 - Deployment Engineering 134
370 Section 291 - Distribution Engineering 138
371 Section 292 - Packaging 142
372 Section 293 - Installation Engineering 146
373 Section 294 - Provisioning 150
374 Section 295 - Environment Promotion 154
375 Section 296 - Feature Management 158
376 Section 297 - Configuration Delivery 162
377 Section 298 - Certificate Management 166
378 Section 299 - Domain And Dns Management 170
379 Section 300 - Content Delivery Networks 174

# 258

## SECTION 258 - CONFIGURATION MANAGEMENT

CODEX/03_SECTIONS/258_CONFIGURATION_MANAGEMENT.txt

SHA-256 5688d7d3a5e33c35a3971c2790c021fd6c5191d9a5ab27ef33dddcdfe1fbe3e6

5,120 BYTES / PRESERVED SOURCE IDENTITY

SECTION 258 — Configuration Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Identify, baseline, change, verify, deploy, observe, and retire configuration that affects system behavior, security, quality, cost, compliance, or recovery.

## REQUIRED OUTCOME

A configuration model with items, owners, source of truth, versions, dependencies, environments, secrets boundary, approval, change history, drift detection, rollback, and evidence.

## WHY THIS MATTERS

Configuration is executable behavior. Untracked flags, defaults, environment values, provider settings, and manual changes create drift that can defeat tests, controls, reproducibility, and recovery.

## APPLICABILITY

Apply when settings, flags, policies, infrastructure, runtime values, schemas, device parameters, or environment-specific behavior can affect an outcome.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no configuration or behavior changes and an approved exact baseline covers the artifact and environment.

## PREREQUISITES AND DEPENDENCIES

Use architecture, source control, secrets, environment, release, operations, security, testing, and recovery baselines. Identify owners, source of truth, permissible variance, and emergency change authority.

## REQUIRED INPUTS

Configuration items, schema, defaults, values, versions, environments, owners, dependencies, secrets references, flags, drift, change records, tests, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve configuration meaning, secure values, change, emergency override, drift tolerance, rollback, and acceptance. Operators record manual intervention and cannot treat undocumented state as baseline.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may compare authorized configuration, identify drift, draft changes and tests, and explain impact. It must not expose secrets, change protected environments, infer safe defaults, or approve or deploy configuration without authority.

## APPLICABLE EXISTING AGENT ROLES

Configuration Manager, Architect, Platform Engineer, Security Auditor, Tester, Verifier, Implementer, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Inventory items; define source and schema; baseline; classify sensitive values; review changes; test; deploy; detect drift; reconcile; handle emergency changes; roll back; archive; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show item, value reference, environment, version, owner, change, impact, evidence, and limitation without exposing secret material. Require human approval for protected, security, safety, cost, and production changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use declarative or versioned configuration, immutable references where appropriate, environment overlays, validation, policy gates, drift detection, safe defaults, rollback, and audit. Keep secret values outside ordinary configuration records.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show baseline, environment, drift, active changes, owner, evidence freshness, dependencies, and authorized approve, promote, reconcile, roll back, quarantine, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Configuration inventory and schema; baseline; change and approval records; drift report; validation and deployment results; rollback, exception, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify item, version, environment, source, change, actor, time, validation, result, and limitation. A file or dashboard snapshot proves only the captured state.

## VERIFICATION AND ACCEPTANCE TESTS

Verify source of truth, schema, defaults, secret boundary, environment promotion, drift, policy, rollback, audit, and recovery. Test missing, invalid, stale, conflicting, and emergency values.

## FAILURE STATES AND ANTI-PATTERNS

Manual production edits, configuration in secrets or source without boundary, undocumented flags, environment drift, default allow, and rollback that restores code but not configuration.

## RECOVERY AND RECONSIDERATION

Freeze drift, preserve state, compare to last valid baseline, roll back or reconcile, rotate exposed values, and reassess impact. Reopen on environment, provider, requirement, dependency, or policy change.

## COMPLETION CRITERIA

Items, schemas, owners, versions, environments, secrets, changes, drift, tests, evidence, recovery, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve baselines, changes, approvals, drift, incidents, evidence, exceptions, and open gaps. Hand off to Source Control, Build, Release, Platform, Operations, Security, Testing, and Recovery.

## RELATED CODEX SECTIONS

258 Configuration Management; 259 Source Control; 263 Environment Management; 264-268 Build and Delivery; 270 DevSecOps; 271 GitOps; 301-330 Operations.

CODEX/03_SECTIONS/258_CONFIGURATION_MANAGEMENT.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, architecture, delivery, and operations context, and the integration roadmap. Exact configuration items, environments, secrets, approval, drift, and acceptance authority remain unresolved.

# 259

## SECTION 259 - SOURCE CONTROL MANAGEMENT

CODEX/03_SECTIONS/259_SOURCE_CONTROL_MANAGEMENT.txt

SHA-256 11e43d5a8484283d5107cded3e4c763cf2a4b427f3e47e6bdf7a6a1daa422d45

5,338 BYTES / PRESERVED SOURCE IDENTITY

SECTION 259 — Source Control Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Preserve the identity, history, collaboration, review, authorization, integrity, and recoverability of source, configuration, documentation, infrastructure, and other controlled change.

## REQUIRED OUTCOME

A source-control model with repositories, ownership, branches, commits, review, access, signing, protection, provenance, retention, release linkage, recovery, and retirement.

## WHY THIS MATTERS

Source history is evidence and an execution input. Unreviewed changes, lost provenance, exposed repositories, force pushes, generated material, or weak recovery can compromise delivery and accountability.

## APPLICABILITY

Apply when versioned files, code, configuration, documentation, infrastructure, tests, models, data definitions, or artifacts are collaboratively changed or released.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no controlled source, history, review, release, or recovery claim is affected and an approved exact baseline covers the work.

## PREREQUISITES AND DEPENDENCIES

Use identity, access, IP, licensing, secrets, build, release, records, backup, and security baselines. Identify repository owners, branch policy, review authority, retention, and emergency change.

## REQUIRED INPUTS

Repositories, content, authors, ownership, branches, commits, reviews, protections, dependencies, secrets scan, release linkage, access, retention, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve repository policy, protected changes, reviewers, access, emergency bypass, retention, and deletion. Humans review meaning, security, IP, and release risk; authors remain accountable for truthful change description.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized history, draft changes, tests, messages, and review summaries. It must preserve authorship and provenance, not impersonate approval, rewrite history, expose secrets, merge, delete, or bypass branch protection without explicit authority.

## APPLICABLE EXISTING AGENT ROLES

Source Control Manager, Implementer, Reviewer, Security Auditor, IP Analyst, Tester, Verifier, Researcher, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Inventory repositories; assign owners; define access and protection; capture changes; review; scan; link to issues, builds, and releases; back up; recover; handle emergency changes; audit; and retire repositories with records.

## HUMAN-AI INTERACTION REQUIREMENTS

Show repository, ref, author, change, review, protection, evidence, and intended effect. Require human approval for merge, protected branch bypass, secret or IP handling, release, and deletion.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use immutable or auditable history, protected branches, signed commits or equivalent provenance where warranted, review gates, access segregation, secret scanning, backup, and release traceability.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show repository health, owners, changes, reviews, protection, findings, release links, evidence freshness, and authorized approve, reject, merge, revert, restrict, recover, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Repository inventory and policy; access and protection model; review and provenance record; scan and build links; backup and recovery; exception, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify repository, ref, commit, author, reviewer, time, source, change, method, result, and limitation. A commit proves history, not correctness, authorization, or safe deployment.

## VERIFICATION AND ACCEPTANCE TESTS

Verify access, protection, review, provenance, secret and license scanning, revert, backup, restore, release linkage, audit, and emergency change. Test unauthorized push, lost branch, compromised account, and recovery.

## FAILURE STATES AND ANTI-PATTERNS

Shared accounts, force push without authority, unreviewed protected changes, secrets in history, source without owner, generated code without provenance, and repository deletion without records.

## RECOVERY AND RECONSIDERATION

Restrict compromised access, preserve history, rotate secrets, revert or restore known-good refs, investigate, notify owners, and reassess downstream artifacts. Reopen on source, author, dependency, policy, or release change.

## COMPLETION CRITERIA

Repositories, owners, access, history, review, protection, provenance, releases, recovery, evidence, exceptions, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve repository identity, refs, history, reviews, findings, releases, decisions, backups, evidence, exceptions, and open gaps. Hand off to Build, Release, Security, IP, Licensing, Records, Platform, and Operations.

## RELATED CODEX SECTIONS

223-224 Intellectual Property and Licensing; 258 Configuration; 260-262 Dependencies, Packages, and Artifacts; 264-271 Build, CI/CD, DevOps, DevSecOps, and GitOps; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, source, IP, security, and delivery context, and the integration roadmap. Exact repository, access, branch, review, signing, retention, recovery, and acceptance authority remain unresolved.

# 260

## SECTION 260 - DEPENDENCY MANAGEMENT

CODEX/03_SECTIONS/260_DEPENDENCY_MANAGEMENT.txt

SHA-256 0a9bb3c294732536a408244be70161cd23c2930e2757285b6dc5cbd0606c874c

5,179 BYTES / PRESERVED SOURCE IDENTITY

SECTION 260 — Dependency Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Discover, evaluate, select, version, update, monitor, constrain, and retire dependencies that affect behavior, security, quality, licensing, cost, availability, or trust.

## REQUIRED OUTCOME

A dependency inventory and lifecycle with owner, purpose, version, provenance, license, vulnerability, support, compatibility, update policy, fallback, evidence, and retirement.

## WHY THIS MATTERS

Dependencies are part of the real system even when teams do not own them. Unknown, stale, transitive, unmaintained, or incompatible dependencies can introduce risk that local tests and source review miss.

## APPLICABILITY

Apply when software, hardware, data, service, provider, model, tool, package, API, or process relies on an external or separately owned component.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no external or separately versioned dependency affects the artifact and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use architecture, source, package, license, vulnerability, security, privacy, release, compatibility, support, and operations baselines. Identify dependency owner and acceptable substitution.

## REQUIRED INPUTS

Dependency graph, versions, source, owner, purpose, license, support, vulnerability, compatibility, data access, provider, update, fallback, cost, and retirement status.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve dependency selection, version, risk, license, update, exception, provider, fallback, and retirement. Humans decide whether a change is safe for users and commitments.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized graphs, advisories, changelogs, licenses, and test evidence; identify drift; and draft update or replacement plans. It must not install, upgrade, remove, or accept a dependency without authority or hide transitive impact.

## APPLICABLE EXISTING AGENT ROLES

Dependency Manager, Architect, Security Auditor, License Analyst, Researcher, Analyst, Tester, Verifier, Implementer, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Inventory direct and transitive dependencies; capture provenance and ownership; assess risk, license, compatibility, and support; pin or constrain; monitor advisories; update; test; mitigate; substitute; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show dependency, version, source, owner, license, data, vulnerability, change, tests, evidence, and limitation. Require human approval for install, upgrade, removal, exception, license, and production impact.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Support lockfiles, provenance, SBOM, isolation, version negotiation, compatibility, fallback, patching, rollback, provider substitution, and observability of dependency behavior.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show graph, owners, versions, findings, support, licenses, updates, evidence freshness, impact, and authorized approve, update, pin, replace, isolate, rollback, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Dependency inventory and graph; risk, license, and support assessment; update and fallback plan; SBOM or equivalent; test, exception, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify dependency, version, source, build, environment, license, advisory, method, time, result, owner, and limitation. Package presence or lockfile proves identity, not safety or compatibility.

## VERIFICATION AND ACCEPTANCE TESTS

Verify direct and transitive inventory, provenance, licenses, vulnerabilities, compatibility, behavior, update, rollback, fallback, data access, and retirement. Test unavailable or compromised provider.

## FAILURE STATES AND ANTI-PATTERNS

Unknown transitive dependency, floating version, unreviewed update, stale support, license conflict, advisory ignored, provider single point, and dependency changed without test or rollback.

## RECOVERY AND RECONSIDERATION

Quarantine affected dependency or artifact, roll back or substitute, rotate exposed secrets, notify owners, preserve evidence, and reassess downstream impact. Reopen on advisory, provider, version, license, or architecture change.

## COMPLETION CRITERIA

Dependencies, versions, owners, purpose, provenance, license, risk, support, tests, fallback, evidence, exceptions, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve graph, versions, advisories, decisions, licenses, tests, incidents, evidence, exceptions, and open gaps. Hand off to Package, Build, Security, Licensing, Release, Platform, and Operations.

## RELATED CODEX SECTIONS

223-224 IP and Licensing; 259 Source Control; 261 Package Management; 262 Artifact Management; 264-268 Build and Delivery; 380-381 Security and Dependency Updates; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, source, security, licensing, and delivery context, and the integration roadmap. Exact graph, support, SBOM, policy, providers, update, fallback, and acceptance authority remain unresolved.

# 261

## SECTION 261 - PACKAGE MANAGEMENT

CODEX/03_SECTIONS/261_PACKAGE_MANAGEMENT.txt

SHA-256 7971b85253bcb4b730abd65b6989ddd6cae17c8ef2233082064735c3d0022b70

5,113 BYTES / PRESERVED SOURCE IDENTITY

SECTION 261 — Package Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Govern the acquisition, storage, resolution, publication, promotion, integrity, vulnerability, licensing, and retirement of packages used to build or operate the system.

## REQUIRED OUTCOME

A package-management model with registries, repositories, provenance, resolution, lock or pin policy, access, signing, scanning, retention, promotion, rollback, and owners.

## WHY THIS MATTERS

Packages are executable supply-chain inputs. A mutable registry, malicious artifact, dependency confusion, unsigned package, or unreviewed resolution can alter behavior before local source review.

## APPLICABILITY

Apply when packages, modules, libraries, plugins, containers, firmware bundles, datasets, models, or provider components are fetched, built, cached, promoted, or consumed.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no package or registry input is used and an approved exact baseline covers the artifact.

## PREREQUISITES AND DEPENDENCIES

Use dependency, source, artifact, security, license, secrets, build, release, and network baselines. Identify registries, trust, ownership, retention, and emergency authority.

## REQUIRED INPUTS

Package graph, registry, namespace, version, digest, provenance, license, advisory, policy, credentials, cache, lockfile, promotion, and retirement requirements.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve registries, namespaces, packages, trust, publishing, promotion, exceptions, retention, and withdrawal. Humans resolve identity and license ambiguity and accept supply-chain risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized package metadata, graph, advisories, and lockfiles; draft updates and policies; and identify anomalies. It must not install, publish, promote, delete, or trust a package without authority and verification.

## APPLICABLE EXISTING AGENT ROLES

Package Manager, Supply Chain Analyst, Security Auditor, Dependency Manager, Build Engineer, Researcher, Tester, Verifier, Governance Reviewer, and Documentation Writer.

## REQUIRED ACTIVITIES

Define approved sources; inventory and pin packages; verify provenance and integrity; scan license and vulnerability; isolate and cache; review and promote; monitor registry; withdraw or replace; retain evidence; and recover.

## HUMAN-AI INTERACTION REQUIREMENTS

Show package, registry, namespace, version, digest, provenance, policy, scan, evidence, and limitation. Require human approval for install, publish, promotion, exception, and withdrawal.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use trusted registries, namespace controls, digest or version pinning, signed provenance, isolated builders, cache integrity, allowlists, SBOM, quarantine, and rollback.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show registry, package, version, digest, findings, provenance, consumers, evidence freshness, and authorized approve, quarantine, promote, withdraw, replace, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Package policy and inventory; registry and trust model; lock and provenance records; scan and promotion results; withdrawal and recovery; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify package, registry, namespace, version, digest, source, builder, scan, time, actor, result, and limitation. Name and version alone do not prove artifact identity or safety.

## VERIFICATION AND ACCEPTANCE TESTS

Verify source, namespace, integrity, provenance, license, vulnerability, resolution, cache, promotion, rollback, withdrawal, and consumer impact. Test unavailable, substituted, or malicious registry response.

## FAILURE STATES AND ANTI-PATTERNS

Untrusted registry, dependency confusion, mutable tag, unsigned package, cache poisoning, package installed directly in production, and scan passed without exact digest.

## RECOVERY AND RECONSIDERATION

Quarantine package and consumers, preserve provenance, rotate credentials, roll back or replace, notify owners, and assess downstream artifacts. Reopen on registry, package, advisory, policy, or build change.

## COMPLETION CRITERIA

Packages, registries, trust, versions, digests, owners, scans, promotion, recovery, evidence, exceptions, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve package and digest history, registry policy, provenance, scans, promotions, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Dependency, Build, Artifact, Security, Release, and Platform.

## RELATED CODEX SECTIONS

259 Source Control; 260 Dependency Management; 262 Artifact Management; 264-271 Build, CI/CD, DevOps, DevSecOps, and GitOps; 209 Vulnerability Management; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, supply-chain, security, licensing, and delivery context, and the integration roadmap. Exact registries, provenance, package policy, signing, retention, providers, and acceptance authority remain unresolved.

# 262

## SECTION 262 - ARTIFACT MANAGEMENT

CODEX/03_SECTIONS/262_ARTIFACT_MANAGEMENT.txt

SHA-256 52e67e5dc4d2b47eb09c784340f65e900dd668baa8586ee27414817688b0d7e6

5,077 BYTES / PRESERVED SOURCE IDENTITY

SECTION 262 — Artifact Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Preserve the identity, provenance, integrity, storage, promotion, distribution, retention, and retirement of artifacts produced or consumed across the lifecycle.

## REQUIRED OUTCOME

An artifact model with type, version, digest, source, build, dependencies, environment, owner, status, approval, consumers, retention, evidence, rollback, and disposal.

## WHY THIS MATTERS

Artifacts are what systems actually deploy, distribute, analyze, or archive. A source commit or successful build does not prove that the shipped artifact is the same, complete, authorized, or safe.

## APPLICABILITY

Apply to binaries, packages, containers, firmware, models, datasets, reports, documentation, infrastructure plans, test evidence, and other lifecycle outputs.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no artifact is retained, promoted, consumed, or used as evidence and an approved exact baseline covers the work.

## PREREQUISITES AND DEPENDENCIES

Use source, package, dependency, build, security, license, release, environment, records, retention, and recovery baselines. Identify repository, registry, owner, and promotion authority.

## REQUIRED INPUTS

Artifact type, source, version, digest, build, dependencies, metadata, owner, environment, approval, scan, consumers, retention, distribution, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve artifact status, promotion, distribution, retention, withdrawal, exception, and deletion. Humans verify provenance and consequences before release or use.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized metadata, compare artifacts, draft manifests and checks, and identify anomalies. It must not fabricate provenance, promote, alter, replace, publish, or delete artifacts without explicit authority.

## APPLICABLE EXISTING AGENT ROLES

Artifact Manager, Build Engineer, Release Engineer, Security Auditor, Dependency Manager, Tester, Verifier, Governance Reviewer, and Documentation Writer.

## REQUIRED ACTIVITIES

Define artifact classes; capture metadata and provenance; hash and sign where required; scan; store; approve; promote; distribute; monitor; revoke or replace; back up; retain; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show artifact, digest, source, build, dependency, environment, status, owner, evidence, and limitation. Require human approval for promotion, distribution, withdrawal, retention exception, and deletion.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use immutable storage, content addressing, provenance, SBOM, access control, quarantine, promotion stages, retention, replication, rollback, and consumer impact tracking.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show artifact identity, status, provenance, consumers, findings, evidence freshness, retention, owner, and authorized approve, promote, quarantine, withdraw, restore, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Artifact inventory and schema; provenance and integrity record; storage and promotion model; scan and approval; distribution and rollback; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify artifact, digest, source, build, dependencies, environment, signer, method, time, result, actor, and limitation. Filename, tag, or registry path alone proves neither identity nor integrity.

## VERIFICATION AND ACCEPTANCE TESTS

Verify identity, provenance, digest, signature, metadata, scan, access, promotion, distribution, rollback, withdrawal, retention, and recovery. Test corruption, replacement, provider loss, and consumer mismatch.

## FAILURE STATES AND ANTI-PATTERNS

Mutable artifact, missing digest, build output not retained, artifact not linked to source, unscanned release, broad registry access, and retirement without consumer impact analysis.

## RECOVERY AND RECONSIDERATION

Quarantine or withdraw, preserve evidence, restore known-good artifact, notify consumers, rotate trust if needed, and reassess releases and records. Reopen on source, build, dependency, policy, or consumer change.

## COMPLETION CRITERIA

Artifacts, identity, provenance, owners, status, consumers, storage, promotion, evidence, recovery, retention, exceptions, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve artifact and digest history, build links, consumers, promotions, findings, decisions, evidence, exceptions, and open gaps. Hand off to Build, Release, Package, Security, Records, Platform, and Operations.

## RELATED CODEX SECTIONS

259-261 Source, Dependency, and Package Management; 264-268 Build and Delivery; 388 Version Management; 391-395 Data Lifecycle; 397-400 Decommissioning.

CODEX/03_SECTIONS/262_ARTIFACT_MANAGEMENT.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, supply-chain, release, and evidence context, and the integration roadmap. Exact artifact classes, signing, storage, retention, consumers, distribution, and acceptance authority remain unresolved.

# 263

## SECTION 263 - ENVIRONMENT MANAGEMENT

CODEX/03_SECTIONS/263_ENVIRONMENT_MANAGEMENT.txt

SHA-256 d2c466e7ed5e19a6940799ec5f975b758029eb21e3bdfbe5b298e69018ded235

5,247 BYTES / PRESERVED SOURCE IDENTITY

SECTION 263 — Environment Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Define, provision, secure, observe, change, reconcile, and retire environments in which development, testing, staging, production, field, recovery, or archival work occurs.

## REQUIRED OUTCOME

An environment model with purpose, owner, identity, resources, configuration, data, dependencies, isolation, access, lifecycle, evidence, cost, drift, recovery, and promotion rules.

## WHY THIS MATTERS

Environment differences create false tests, unsafe releases, data exposure, and unreproducible incidents. An environment is more than infrastructure; it includes data, identity, providers, configuration, people, and operating mode.

## APPLICABILITY

Apply whenever work is developed, tested, deployed, operated, recovered, or retired in a distinct technical, physical, organizational, or data context.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no environment boundary, configuration, data, access, or operational behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use configuration, identity, network, data, security, test, build, release, cost, operations, and recovery baselines. Define environment owner, purpose, isolation, and data authority.

## REQUIRED INPUTS

Environment purpose, resources, versions, configuration, data, access, provider, dependencies, network, secrets, telemetry, cost, lifecycle, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve creation, access, data, changes, promotion, cost, isolation, emergency use, and retirement. Operators maintain truthful state; humans decide production or sensitive-data use.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized environment metadata, compare drift, draft infrastructure or validation, and summarize health. It must not provision, alter, access sensitive data, promote, or destroy environments without authority.

## APPLICABLE EXISTING AGENT ROLES

Environment Manager, Platform Engineer, Architect, Security Auditor, Data Engineer, Tester, Verifier, Implementer, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define purpose and boundary; inventory resources and data; provision; secure access; configure; validate parity and isolation; monitor drift and cost; promote or clone; recover; rotate; and retire with records.

## HUMAN-AI INTERACTION REQUIREMENTS

Show environment, purpose, owner, resources, data, identity, configuration, provider, evidence, and impact. Require human approval for sensitive data, production, security, cost, and destruction actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use isolated accounts or boundaries, infrastructure as code where appropriate, synthetic data, environment overlays, identity separation, observability, backup, restore, and controlled promotion.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show environment state, drift, resources, data classification, access, cost, incidents, evidence freshness, owner, and authorized create, restrict, reconcile, promote, restore, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Environment catalog; purpose and boundary record; resource and configuration inventory; access and data plan; drift and cost reports; promotion, recovery, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify environment, purpose, version, configuration, resource, data, identity, provider, method, time, result, owner, and limitation. A provisioning script proves intended state, not actual state.

## VERIFICATION AND ACCEPTANCE TESTS

Verify isolation, access, data, configuration, provider, parity, drift, observability, cost, backup, restore, promotion, and retirement. Test accidental cross-environment access and provider loss.

## FAILURE STATES AND ANTI-PATTERNS

Production as test, copied sensitive data, shared credentials, undocumented manual change, drift ignored, environment assumed identical, and destroyed environment without records or recovery.

## RECOVERY AND RECONSIDERATION

Restrict affected environment, preserve state, restore or rebuild from known-good baseline, rotate credentials, notify owners, and reassess data and release impact. Reopen on boundary, provider, configuration, data, or purpose change.

## COMPLETION CRITERIA

Purpose, boundary, owner, resources, data, identity, configuration, dependencies, cost, evidence, recovery, lifecycle, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve environment identity, state, configuration, access, data, changes, incidents, backups, decisions, evidence, exceptions, and open gaps. Hand off to Build, Release, Platform, Security, Data, Testing, and Operations.

## RELATED CODEX SECTIONS

029 Operational Architecture; 258 Configuration; 264-272 Build, CI/CD, DevOps, DevSecOps, GitOps, and Platform; 301-330 Operations and Recovery.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, architecture, delivery, security, data, and operations context, and the integration roadmap. Exact environments, providers, data, isolation, cost, recovery, and acceptance authority remain unresolved.

# 264

## SECTION 264 - BUILD ENGINEERING

CODEX/03_SECTIONS/264_BUILD_ENGINEERING.txt

SHA-256 c25be6a79d6e953559b2f82e0963ea74e5a209c731986709ab0a949a1d1c02bf

5,164 BYTES / PRESERVED SOURCE IDENTITY

SECTION 264 — Build Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design reproducible, secure, observable, and maintainable processes that transform controlled source, dependencies, configuration, and tools into identified artifacts.

## REQUIRED OUTCOME

A build model with inputs, toolchain, environment, dependency resolution, provenance, reproducibility, security, caching, outputs, failure, evidence, and owner control.

## WHY THIS MATTERS

The build is a supply-chain boundary. Nonreproducible tools, mutable inputs, secret leakage, hidden network access, or untracked generated files can make an artifact impossible to trust or recover.

## APPLICABILITY

Apply when source, packages, configuration, models, data, firmware, infrastructure, or other inputs are transformed into a distributable or deployable artifact.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no transformation, artifact, dependency, toolchain, or provenance claim is affected and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use source, dependency, package, artifact, environment, secrets, security, license, testing, and release baselines. Identify build owner, trust, runner, and output authority.

## REQUIRED INPUTS

Source ref, dependencies, packages, toolchain, configuration, environment, secrets references, scripts, inputs, outputs, cache, provenance, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Build owners approve toolchain, inputs, runners, network, secrets, output, reproducibility, exceptions, and release linkage. Humans decide whether a nonreproducible or emergency build is usable.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized build definitions and logs, draft improvements, analyze failures, and propose hardening. It must not run untrusted code with privileged access, expose secrets, modify build gates, or claim provenance without verification.

## APPLICABLE EXISTING AGENT ROLES

Build Engineer, Supply Chain Analyst, Security Auditor, Dependency Manager, Platform Engineer, Tester, Verifier, Implementer, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define inputs and toolchain; pin dependencies; isolate runner; secure credentials; build; capture provenance and artifacts; verify reproducibility; scan; manage cache; handle failure; promote or reject; and retire toolchain.

## HUMAN-AI INTERACTION REQUIREMENTS

Show source, toolchain, runner, dependency, configuration, network, secret boundary, artifact, result, and limitation. Require human approval for privileged build, gate change, emergency output, and promotion.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use isolated and ephemeral runners, hermetic or declared network, provenance, SBOM, cache integrity, reproducible inputs, least privilege, artifact signing, and rollback.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show build, source, toolchain, runner, dependencies, findings, provenance, artifact, evidence freshness, and authorized start, cancel, retry, quarantine, approve, promote, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Build specification; toolchain and runner record; input and provenance manifest; logs; artifact and scan; reproducibility result; exception, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify source, toolchain, runner, dependencies, configuration, time, actor, method, artifact digest, result, and limitation. A successful build proves execution, not safe or reproducible inputs.

## VERIFICATION AND ACCEPTANCE TESTS

Verify isolation, dependencies, secrets, network, reproducibility, provenance, artifact identity, scans, cache, failure, cleanup, and rollback. Repeat from a clean environment.

## FAILURE STATES AND ANTI-PATTERNS

Mutable toolchain, hidden network fetch, privileged runner, secrets in logs, untracked generated output, cache trust, nonreproducible build, and green build treated as release approval.

## RECOVERY AND RECONSIDERATION

Quarantine output, preserve logs, restore trusted toolchain, rotate exposed secrets, rebuild, compare artifacts, and reassess consumers. Reopen on source, dependency, toolchain, runner, or configuration change.

## COMPLETION CRITERIA

Inputs, toolchain, runner, dependencies, outputs, provenance, reproducibility, security, evidence, owners, exceptions, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve build definitions, toolchains, runners, logs, artifacts, provenance, findings, decisions, evidence, exceptions, and open gaps. Hand off to Automation, CI/CD, Artifact, Security, Release, Platform, and Operations.

## RELATED CODEX SECTIONS

259-263 Source, Dependency, Package, Artifact, and Environment Management; 265-271 Build Automation, CI/CD, DevOps, DevSecOps, and GitOps; 388 Version Management.

CODEX/03_SECTIONS/264_BUILD_ENGINEERING.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, supply-chain, build, security, and delivery context, and the integration roadmap. Exact toolchain, runner, reproducibility, provenance, secret, network, and acceptance authority remain unresolved.

# 265

## SECTION 265 - BUILD AUTOMATION

CODEX/03_SECTIONS/265_BUILD_AUTOMATION.txt

SHA-256 7e8ffcd5a55e6763b81e55bf878d93580f29d8b656b457d4b9819c7627dfa93a

4,786 BYTES / PRESERVED SOURCE IDENTITY

SECTION 265 — Build Automation

## STATUS: CONTENT-COMPLETE

## PURPOSE

Automate repeatable build orchestration, validation, artifact production, evidence capture, and safe failure handling without hiding authority or supply-chain risk.

## REQUIRED OUTCOME

A build-automation model with triggers, inputs, runners, steps, credentials, caches, outputs, logs, retries, cancellation, ownership, provenance, and maintenance.

## WHY THIS MATTERS

Automation can scale errors and privilege. A script that “usually works” may silently fetch mutable inputs, leak secrets, retry unsafe actions, or generate artifacts with no trustworthy record.

## APPLICABILITY

Apply when build or artifact creation is scripted, scheduled, event-driven, or run by CI, a platform, or an agent.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no build automation, artifact, privileged step, or evidence claim changes and an approved exact baseline applies.

## PREREQUISITES AND DEPENDENCIES

Use build, source, dependency, package, artifact, runner, secret, security, testing, and release baselines. Identify automation owner and stop authority.

## REQUIRED INPUTS

Workflow, trigger, source, dependencies, runner, credentials, cache, steps, outputs, logs, retry, cancellation, gates, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve triggers, privilege, credentials, gates, retries, release linkage, and workflow changes. Humans retain control over production or destructive steps.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft workflows, analyze logs, suggest tests, and identify unsafe permissions. It must not activate, expand, or modify privileged automation or approve output without explicit human control.

## APPLICABLE EXISTING AGENT ROLES

Build Automation Engineer, Build Engineer, Platform Engineer, Security Auditor, Tester, Verifier, Implementer, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define workflow and trust; isolate runner; validate input; execute; capture logs and provenance; handle retry and cancel; scan; publish or quarantine; test failure; rotate credentials; maintain and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show workflow, trigger, identity, privilege, source, runner, step, output, log, and limitation. Require human approval for privileged or externally visible actions and gate changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use least privilege, ephemeral runners, secret injection, immutable artifacts, idempotent steps, bounded retries, cancellation, audit, concurrency controls, and failure isolation.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show workflow status, actor, source, runner, active step, output, findings, evidence freshness, and authorized run, cancel, retry, approve, quarantine, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Workflow definition; permissions and runner record; execution log; artifact and provenance; retry and failure record; test, decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify workflow version, trigger, actor, runner, source, step, time, output, result, and limitation. A green workflow proves only that execution and declared coverage.

## VERIFICATION AND ACCEPTANCE TESTS

Verify permissions, trigger, input validation, secret masking, isolation, retries, cancellation, artifact identity, logs, cleanup, and failure recovery. Test malicious input and runner loss.

## FAILURE STATES AND ANTI-PATTERNS

Overprivileged runner, secrets in output, unbounded retry, trigger surprise, mutable action, skipped gate, non-idempotent step, and workflow ownership without maintenance.

## RECOVERY AND RECONSIDERATION

Cancel, isolate runner, preserve logs, rotate credentials, quarantine output, restore trusted workflow, and reassess downstream impact. Reopen on workflow, trigger, permission, tool, or artifact change.

## COMPLETION CRITERIA

Workflow, trigger, permissions, runners, steps, artifacts, evidence, failure, owners, maintenance, exceptions, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve workflow versions, permissions, runs, logs, artifacts, findings, decisions, evidence, exceptions, and open gaps. Hand off to CI, CD, Build, Security, Platform, Release, and Operations.

## RELATED CODEX SECTIONS

264 Build Engineering; 266-271 CI, CD, DevOps, DevSecOps, and GitOps; 258 Configuration; 262 Artifacts; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, automation, security, and delivery context, and the integration roadmap. Exact workflows, triggers, runners, permissions, credentials, gates, and acceptance authority remain unresolved.

# 266

## SECTION 266 - CONTINUOUS INTEGRATION

CODEX/03_SECTIONS/266_CONTINUOUS_INTEGRATION.txt

SHA-256 26777ddb699e70edcacf312450f8589da15b68760cd6a4a3590ae8a7f4400421

4,742 BYTES / PRESERVED SOURCE IDENTITY

SECTION 266 — Continuous Integration

## STATUS: CONTENT-COMPLETE

## PURPOSE

Integrate controlled changes frequently and provide fast, trustworthy feedback about build, test, quality, security, dependency, and contract health.

## REQUIRED OUTCOME

A CI model with trigger, source, isolation, build, test, scan, artifact, evidence, feedback, gate, ownership, failure, and recovery behavior.

## WHY THIS MATTERS

Fast feedback is valuable only when it is truthful, reproducible, relevant, and acted on. A green pipeline can hide skipped tests, stale dependencies, unsafe permissions, flaky checks, or environment mismatch.

## APPLICABILITY

Apply when changes are integrated through automated or repeatable build and validation workflows before release or deployment.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no integration, build, test, security, artifact, or change feedback claim is made and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use source, build, package, dependency, test, security, artifact, configuration, environment, and governance baselines. Identify branch, trigger, gate, and owner.

## REQUIRED INPUTS

Change, branch, source, dependencies, workflow, runner, tests, scans, artifact, gates, secrets, environment, results, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners define required gates, flaky policy, bypass, merge, and remediation. Humans review important failures and retain authority for protected branch and release decisions.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may summarize runs, diagnose failures, draft tests, and propose fixes. It must not mark checks passed, suppress failures, bypass gates, merge, or change workflow permissions without human authority.

## APPLICABLE EXISTING AGENT ROLES

CI Engineer, Build Engineer, Tester, Security Auditor, Dependency Manager, Quality Engineer, Verifier, Implementer, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Trigger on controlled change; build; test; scan; capture artifacts and evidence; report; triage; manage flaky or quarantined tests; enforce gate; allow controlled retry or bypass; restore; and improve.

## HUMAN-AI INTERACTION REQUIREMENTS

Show change, branch, workflow, runner, checks, failures, evidence, gate, actor, and limitation. Require human approval for bypass, merge, permission, quarantine, and release-impacting actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use isolated runners, repeatable toolchain, secure secrets, parallelism with resource limits, artifact identity, required checks, policy as code, and durable logs.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show run, change, checks, gate, failures, flakiness, artifacts, evidence freshness, owner, and authorized rerun, cancel, quarantine, bypass, merge, and reopen controls.

## REQUIRED OUTPUTS AND ARTIFACTS

CI workflow and policy; run record; build and test results; scan and artifact; gate and bypass record; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify change, ref, workflow, runner, toolchain, check, result, artifact, time, actor, and limitation. Green status proves only completed checks and their coverage.

## VERIFICATION AND ACCEPTANCE TESTS

Verify trigger, isolation, required checks, failure visibility, secret handling, artifacts, reproducibility, flaky policy, bypass audit, and recovery. Test malicious change and runner loss.

## FAILURE STATES AND ANTI-PATTERNS

Green with skipped checks, unbounded bypass, flaky retry theater, shared runner privilege, secrets in logs, CI as release approval, and no owner for failing pipeline.

## RECOVERY AND RECONSIDERATION

Stop affected integration, preserve run, repair or rollback workflow or change, rotate secrets, rerun, and reassess gate. Reopen on tool, dependency, policy, source, or environment change.

## COMPLETION CRITERIA

Triggers, changes, checks, runners, artifacts, gates, owners, evidence, bypass, recovery, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve workflow, runs, checks, artifacts, bypasses, findings, decisions, evidence, exceptions, and open gaps. Hand off to CD, Build, Release, Security, Platform, Testing, and Operations.

## RELATED CODEX SECTIONS

264-265 Build and Build Automation; 267-268 Continuous Delivery and Deployment; 270 DevSecOps; 271 GitOps; 231-255 Testing.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, build, testing, security, and delivery context, and the integration roadmap. Exact workflow, gates, runner, bypass, artifact, secret, and acceptance authority remain unresolved.

# 267

## SECTION 267 - CONTINUOUS DELIVERY

CODEX/03_SECTIONS/267_CONTINUOUS_DELIVERY.txt

SHA-256 e773954191b6f8ced4fd6a2c90aadd263883e9dd3652dd7d5a273d427721410e

5,060 BYTES / PRESERVED SOURCE IDENTITY

SECTION 267 — Continuous Delivery

## STATUS: CONTENT-COMPLETE

## PURPOSE

Keep validated, identified artifacts ready for controlled release through promotion, approval, environment preparation, rollback, and evidence-preserving delivery.

## REQUIRED OUTCOME

A delivery model with artifact identity, readiness gates, environments, approvals, configuration, data migration, communication, release strategy, rollback, and owner decision.

## WHY THIS MATTERS

Delivery is not deployment. A ready artifact can still be unsuitable for an environment, user population, data migration, support capacity, or commitment. Automation must preserve human authority where consequence requires it.

## APPLICABILITY

Apply when artifacts are promoted or prepared for release across environments, customers, operators, devices, or services.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no artifact, environment, user, data, support, or release readiness changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use build, artifact, configuration, environment, testing, acceptance, operations, support, data migration, security, and rollback baselines.

## REQUIRED INPUTS

Artifact, provenance, tests, findings, environment, configuration, migration, readiness, approvals, support, communication, rollback, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Release owners approve readiness, risk, timing, communication, migration, exception, and rollback. Humans decide whether evidence and support capacity justify release.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may assemble readiness evidence, draft plans, identify gaps, and summarize changes. It must not declare ready, approve, promote, communicate commitments, or execute release without delegated authority and human control.

## APPLICABLE EXISTING AGENT ROLES

Release Engineer, Delivery Engineer, Architect, Tester, Verifier, Operations Engineer, Security Auditor, Support Reviewer, Coordinator, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Validate artifact and evidence; assess impact; prepare environment and configuration; plan migration and communication; obtain approvals; promote; monitor; rollback or pause; record outcome; and hand off operations.

## HUMAN-AI INTERACTION REQUIREMENTS

Show artifact, environment, change, evidence, readiness, approval, dependency, user impact, communication, and rollback. Require human approval for release, data migration, exception, and external commitment.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use immutable artifacts, promotion stages, gates, feature flags, canary or phased release, migration checkpoints, rollback, observability, support readiness, and access separation.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show release candidate, readiness, environment, approvals, dependencies, incidents, evidence freshness, communication, rollback, and authorized promote, pause, rollback, restrict, notify, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Delivery plan; readiness checklist; artifact and environment record; migration and communication plan; approvals; monitoring and rollback; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify artifact, environment, configuration, test, approval, actor, time, method, result, impact, and limitation. Readiness evidence does not prove post-release outcome.

## VERIFICATION AND ACCEPTANCE TESTS

Verify artifact identity, gates, approvals, environment, data, support, communication, migration, monitoring, rollback, and recovery. Exercise failed promotion and partial rollout.

## FAILURE STATES AND ANTI-PATTERNS

Deploy without readiness, artifact drift, approval theater, migration without backup, rollback that misses data, support unprepared, and delivery status treated as user success.

## RECOVERY AND RECONSIDERATION

Pause or roll back, preserve state and evidence, communicate, restore or compensate, and reassess users, data, commitments, and residual risk. Reopen on artifact, environment, data, or support change.

## COMPLETION CRITERIA

Artifact, readiness, environment, approvals, migration, support, communication, rollback, evidence, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve release identity, artifacts, approvals, environment, telemetry, incidents, communications, decisions, evidence, exceptions, and open gaps. Hand off to Deployment, Operations, Support, Product, Security, and Governance.

## RELATED CODEX SECTIONS

237 Acceptance Testing; 262 Artifacts; 263 Environments; 266 CI; 268 Continuous Deployment; 274 Release Engineering; 301-330 Operations; 373 Roadmap Management.

CODEX/03_SECTIONS/267_CONTINUOUS_DELIVERY.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, delivery, operations, testing, and owner-control context, and the integration roadmap. Exact release authority, gates, environments, migration, support, rollback, and acceptance evidence remain unresolved.

# 268

## SECTION 268 - CONTINUOUS DEPLOYMENT

CODEX/03_SECTIONS/268_CONTINUOUS_DEPLOYMENT.txt

SHA-256 64d4829b31872a3c7f3121c181b627039f72ae8726f3c1b969cfa11cb2006fb5

5,066 BYTES / PRESERVED SOURCE IDENTITY

SECTION 268 — Continuous Deployment

## STATUS: CONTENT-COMPLETE

## PURPOSE

Deploy validated artifacts through automated, observable, reversible, and authorized paths while protecting users, data, service continuity, and owner control.

## REQUIRED OUTCOME

A deployment model with trigger, artifact, target, identity, policy gates, strategy, configuration, migration, monitoring, rollback, communication, approval, and incident behavior.

## WHY THIS MATTERS

Automation can change production faster than humans can see. Without strong gates, canary, rollback, observability, and authority, a small change can become a broad outage, exposure, or irreversible data event.

## APPLICABILITY

Apply when artifacts are automatically or repeatedly deployed to an environment, device population, customer, service, or infrastructure target.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no deployment, target, configuration, data, or operational behavior changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use delivery, artifact, environment, configuration, identity, security, testing, operations, migration, backup, rollback, and incident baselines. Define deployment and stop authority.

## REQUIRED INPUTS

Artifact, target, version, trigger, identity, gates, configuration, migration, strategy, health, telemetry, rollback, communication, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve automation scope, target, gates, strategy, rollback, exceptions, and kill switch. Humans decide emergency deployment, data migration, service interruption, and recovery tradeoffs.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may assemble deployment evidence, draft manifests, analyze health, and recommend pause or rollback. It must not deploy, expand target, alter gates, suppress health, or declare success without explicit delegation and human control.

## APPLICABLE EXISTING AGENT ROLES

Deployment Engineer, Release Engineer, Platform Engineer, Operations Engineer, Tester, Verifier, Security Auditor, Coordinator, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Validate artifact and target; check gates and identity; deploy phased or canary; observe; pause or roll back on threshold; reconcile configuration and data; communicate; record; and improve.

## HUMAN-AI INTERACTION REQUIREMENTS

Show artifact, target, identity, gate, phase, health, evidence, user impact, rollback, and authority. Require human approval for production, sensitive target, migration, gate bypass, and recovery.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use progressive rollout, feature flags, immutable artifact, health checks, automatic safe pause, independent control path, migration checkpoint, rollback, audit, and target isolation.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show deployment phase, target population, artifact, health, incidents, evidence freshness, owner, rollback state, and authorized pause, stop, rollback, narrow, promote, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Deployment policy and manifest; target and identity record; rollout and health evidence; migration and rollback; incident and communication; decision, exception, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify artifact, target, phase, identity, configuration, health metric, threshold, method, time, result, impact, and limitation. Deployment completion proves execution, not service or user success.

## VERIFICATION AND ACCEPTANCE TESTS

Verify gates, target isolation, canary, health, pause, rollback, migration, audit, access, cleanup, and recovery. Test failed deployment, partial rollout, stale health, and control-plane loss.

## FAILURE STATES AND ANTI-PATTERNS

Broad automatic rollout, no kill switch, health metric blind to user outcome, rollback that cannot reverse data, gate bypass, and deployment success equated with acceptance.

## RECOVERY AND RECONSIDERATION

Pause or roll back, preserve state, communicate, restore or compensate, analyze impact, and reassess release. Reopen on target, artifact, health, configuration, migration, or dependency change.

## COMPLETION CRITERIA

Artifact, target, gates, identity, rollout, health, rollback, owner, evidence, incidents, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve manifests, artifacts, rollout timeline, health, decisions, incidents, communications, evidence, exceptions, and open gaps. Hand off to Operations, Release, Platform, Security, Support, and Incident Response.

## RELATED CODEX SECTIONS

263 Environments; 267 Continuous Delivery; 269-272 DevOps, DevSecOps, GitOps, and Platform Engineering; 301-330 Operations and Recovery.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, delivery, deployment, security, and operations context, and the integration roadmap. Exact targets, gates, health, rollout, rollback, authority, migration, and acceptance evidence remain unresolved.

# 269

## SECTION 269 - DEVOPS

CODEX/03_SECTIONS/269_DEVOPS.txt

SHA-256 0a306e9ecf3ed03fdeb332b5f5c6842b89457201453119a3f488d6e79138eac8

5,254 BYTES / PRESERVED SOURCE IDENTITY

SECTION 269 — DevOps

## STATUS: CONTENT-COMPLETE

## PURPOSE

Integrate development, delivery, operations, security, quality, and learning so teams can change systems quickly while preserving reliability, accountability, and user outcomes.

## REQUIRED OUTCOME

A DevOps operating model with shared ownership, flow, feedback, automation, environments, observability, incident learning, quality, security, and measurable improvement.

## WHY THIS MATTERS

DevOps is not a tool list or a team merger. Without clear authority and protected outcomes, speed can shift work to operators, hide risk, increase fatigue, or optimize deployment frequency over service quality.

## APPLICABILITY

Apply when development and operations jointly create, release, operate, secure, support, or improve a product or service.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no shared delivery, operating, support, security, or feedback workflow is affected and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use product, architecture, quality, security, delivery, operations, support, reliability, incident, and governance baselines. Identify team boundaries, service owners, and decision rights.

## REQUIRED INPUTS

Value and user outcomes, team roles, workflow, source, build, deployment, operations, telemetry, incidents, support, risks, measures, and improvement constraints.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Service and product owners share outcomes but retain explicit authority. Humans set risk, staffing, on-call, change, incident, security, and user-impact decisions and protect sustainable workload.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may summarize flow and operational evidence, draft automation, identify bottlenecks, and propose improvements. It must not erase ownership, change production, suppress incidents, set policy, or optimize delivery speed at the expense of protected outcomes without human decision.

## APPLICABLE EXISTING AGENT ROLES

Architect, Planner, Implementer, Operations Engineer, Security Auditor, Tester, Verifier, Coordinator, Optimizer, User Experience Reviewer, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define outcomes and ownership; map flow; automate safe feedback; integrate testing and security; deploy progressively; observe; operate and support; respond; learn; improve; measure quality and workload; and review architecture and team boundaries.

## HUMAN-AI INTERACTION REQUIREMENTS

Show owner, outcome, change, evidence, risk, operational state, user impact, and authority. Require human control for production, security, incident, staffing, and service commitments. Preserve operator stop and appeal.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Support self-service with guardrails, immutable artifacts, policy gates, observability, ownership metadata, safe deployment, rollback, incident, knowledge, and feedback loops. Avoid central automation becoming a single control failure.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show flow, lead time, change, reliability, incidents, workload, cost, security, evidence freshness, owners, and authorized pause, rollback, remediate, escalate, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

DevOps charter; ownership and service map; flow and automation design; CI/CD and operations model; observability and incident learning; measures, decisions, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify outcome, service, change, environment, measure, period, source, method, actor, result, and limitation. Delivery frequency or automation count is not service quality.

## VERIFICATION AND ACCEPTANCE TESTS

Verify ownership, workflow, security, tests, deployment, observability, support, incident, recovery, workload, and user outcomes. Test team or provider failure and unsafe automation.

## FAILURE STATES AND ANTI-PATTERNS

Tool worship, speed-only metrics, invisible operations work, shared accountability with no owner, automation without rollback, on-call burnout, and incidents treated as individual failure.

## RECOVERY AND RECONSIDERATION

Stabilize service, protect operators and users, preserve evidence, roll back or reduce change, address workload and systemic cause, and revisit measures. Reopen on outcome, organization, service, architecture, or risk change.

## COMPLETION CRITERIA

Outcomes, owners, flow, automation, security, quality, operations, support, measures, evidence, workload, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve service maps, ownership, workflows, runbooks, incidents, decisions, evidence, exceptions, workload assumptions, and open gaps. Hand off to Platform, DevSecOps, GitOps, SRE, Support, Product, and Governance.

## RELATED CODEX SECTIONS

029 Operational Architecture; 258-268 Configuration through Deployment; 270-272 DevSecOps, GitOps, and Platform Engineering; 301-330 Operations; 402-404 Review.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, delivery and operations context, and the integration roadmap. Exact teams, ownership, measures, workload, service objectives, security, and acceptance authority remain unresolved.

# 270

## SECTION 270 - DEVSECOPS

CODEX/03_SECTIONS/270_DEVSECOPS.txt

SHA-256 7ce24b6f97e0972f04e5cf16e8fc8b8b025402ed3cc3d676c911090e8e193e38

5,256 BYTES / PRESERVED SOURCE IDENTITY

SECTION 270 — DevSecOps

## STATUS: CONTENT-COMPLETE

## PURPOSE

Integrate security, privacy, risk, and evidence into development and delivery flow so protected outcomes are considered early, continuously, and with accountable human control.

## REQUIRED OUTCOME

A DevSecOps model with threat and vulnerability input, secure source and build, identity, secrets, testing, policy gates, remediation, monitoring, incident response, and exception lifecycle.

## WHY THIS MATTERS

Security added only at the end is slow and easy to bypass, while automated security checks can create noise, false confidence, or unsafe blocking. Integration must improve decisions, not merely add scanners.

## APPLICABILITY

Apply when software, infrastructure, data, hardware, models, or delivery workflows create material security, privacy, safety, or supply-chain risk.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no security or privacy property, trust boundary, dependency, artifact, or delivery control changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use threat, vulnerability, IAM, cryptography, secrets, privacy, source, dependency, build, testing, release, operations, and incident baselines.

## REQUIRED INPUTS

Threats, assets, source, dependencies, secrets, build, artifacts, environments, security tests, findings, policy gates, owners, exceptions, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Security and service owners define gates, severity, exceptions, remediation, release risk, and response. Humans approve bypass, high-impact changes, disclosure, and residual risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized code, configuration, advisories, scans, and threats; draft fixes and tests; and prioritize findings. It must not exploit, alter security controls, suppress findings, rotate secrets, bypass gates, or approve risk without authority.

## APPLICABLE EXISTING AGENT ROLES

DevSecOps Engineer, Security Auditor, Threat Modeler, Vulnerability Analyst, Build Engineer, Tester, Verifier, Privacy Reviewer, Implementer, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Threat model changes; scan source, dependencies, artifacts, and infrastructure; test security; protect secrets; enforce policy; triage and remediate; manage exceptions; verify; monitor; respond; and improve controls.

## HUMAN-AI INTERACTION REQUIREMENTS

Show finding, source, asset, version, severity, evidence, owner, gate, recommendation, and limitation. Require human review for exploit, remediation, bypass, disclosure, exception, and release decisions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use policy as code with human override, least privilege, secure runners, secret isolation, provenance, threat traceability, safe fail behavior, alert quality, and rollback. Do not let a security pipeline become a single point of unsafe denial.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show security posture, findings, gates, exceptions, evidence freshness, remediation, deployment impact, owner, and authorized pause, remediate, waive, escalate, release, and reopen controls.

## REQUIRED OUTPUTS AND ARTIFACTS

DevSecOps charter; threat and control map; secure pipeline; scan and test results; finding and exception register; remediation and release decision; evidence and continuity records.

## EVIDENCE REQUIREMENTS

Identify asset, source, version, tool, rule, finding, coverage, environment, method, time, result, owner, and limitation. A clean scan proves only the engine, configuration, and coverage.

## VERIFICATION AND ACCEPTANCE TESTS

Verify source, dependency, secret, build, artifact, infrastructure, identity, privacy, policy, gate, remediation, exception, and incident behavior. Test false positives, malicious input, scanner outage, and bypass audit.

## FAILURE STATES AND ANTI-PATTERNS

Scanner count as security, unowned findings, blocking without risk context, bypass without record, secrets in pipeline, security added too late, and automation trusted more than evidence.

## RECOVERY AND RECONSIDERATION

Quarantine or roll back, preserve evidence, rotate secrets, remediate or mitigate, notify owners, and reassess release and user impact. Reopen on threat, tool, policy, dependency, source, or environment change.

## COMPLETION CRITERIA

Threats, controls, pipeline, findings, owners, gates, exceptions, evidence, remediation, response, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending independent review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve pipeline, findings, scans, decisions, exceptions, evidence, incidents, and open gaps. Hand off to Security Operations, Vulnerability, Threat Modeling, Build, Release, Platform, Privacy, and Governance.

## RELATED CODEX SECTIONS

198-218 Embedded through Privacy; 258-268 Configuration through Deployment; 269 DevOps; 271 GitOps; 301-330 Operations; 386 Technical Debt; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, security, privacy, supply-chain, and delivery context, and the integration roadmap. Exact gates, tools, risk thresholds, exceptions, providers, and acceptance authority remain unresolved.

# 271

## SECTION 271 - GITOPS

CODEX/03_SECTIONS/271_GITOPS.txt

SHA-256 70d40490aa3433b4e1af86c89f70aeaea178d259492f53d407afb25bc2d0e2c7

5,173 BYTES / PRESERVED SOURCE IDENTITY

SECTION 271 — GitOps

## STATUS: CONTENT-COMPLETE

## PURPOSE

Use versioned, reviewed, declarative source as the controlled expression of environment state, with reconciliation, drift visibility, safe promotion, and human authority.

## REQUIRED OUTCOME

A GitOps model with desired state, repository, controller, identity, policy, environment, reconciliation, drift, secrets, promotion, rollback, audit, and recovery.

## WHY THIS MATTERS

Declarative control improves traceability but can make a bad commit or controller rapidly widespread. Desired state is not automatically safe, current, authorized, or complete.

## APPLICABILITY

Apply when versioned source drives infrastructure, platform, application, policy, configuration, or environment reconciliation.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no declarative source, reconciler, drift, or automated environment change is used and an approved exact baseline applies.

## PREREQUISITES AND DEPENDENCIES

Use source control, configuration, identity, policy, secrets, build, artifact, environment, platform, security, and operations baselines. Define repository and reconciler authority.

## REQUIRED INPUTS

Desired state, repository, review, controller, target, identity, policy, artifact, secrets, drift, health, rollback, and acceptance criteria.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve repository, protected changes, controller permissions, promotion, emergency override, drift reconciliation, and rollback. Humans decide whether desired state is safe and correct.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft declarative changes, compare desired and actual state, explain drift, and propose remediation. It must not commit, merge, reconcile, change controller permissions, expose secrets, or declare health without authority.

## APPLICABLE EXISTING AGENT ROLES

GitOps Engineer, Platform Engineer, Source Control Manager, Configuration Manager, Security Auditor, Tester, Verifier, Implementer, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define desired state; protect repository; validate and scan; review and promote; configure controller; reconcile; observe drift and health; pause or roll back; handle emergency changes; restore; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show commit, desired state, target, controller, permissions, drift, health, evidence, and limitation. Require human approval for protected change, reconcile, production, emergency, and destructive action.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate desired source, controller, target, secrets, policy, observed state, health, and recovery. Use least privilege, progressive promotion, drift policy, reconciliation pause, immutable artifacts, and independent control path.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show desired and actual state, commit, target, controller, drift, health, evidence freshness, owner, and authorized approve, pause, reconcile, rollback, override, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

GitOps charter; repository and controller model; desired-state definitions; policy and secret boundaries; promotion and drift records; health, rollback, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify commit, desired state, target, controller, identity, time, observed state, method, result, drift, and limitation. A commit proves intended state, not actual state or safe reconciliation.

## VERIFICATION AND ACCEPTANCE TESTS

Verify repository protection, policy, controller permission, secret handling, drift, reconciliation, health, pause, rollback, recovery, and audit. Test malicious commit, controller loss, and conflicting manual change.

## FAILURE STATES AND ANTI-PATTERNS

Controller overprivilege, automatic broad reconciliation, secrets in repository, drift hidden, manual change overwritten, desired state mistaken for reality, and no emergency stop.

## RECOVERY AND RECONSIDERATION

Pause reconciliation, preserve commit and state, restore known-good desired and actual state, rotate credentials, notify owners, and reassess blast radius. Reopen on source, controller, target, policy, or environment change.

## COMPLETION CRITERIA

Desired state, repository, controller, target, identity, policy, drift, health, rollback, evidence, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve repositories, commits, controller state, desired and actual state, promotions, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Platform, Configuration, Build, Release, Security, and Operations.

## RELATED CODEX SECTIONS

258 Configuration; 259 Source Control; 263 Environment; 264-268 Build and Delivery; 269-270 DevOps and DevSecOps; 272 Platform Engineering; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, declarative delivery, security, and operations context, and the integration roadmap. Exact repository, controller, permissions, drift, secret, target, rollback, and acceptance authority remain unresolved.

# 272

## SECTION 272 - PLATFORM ENGINEERING

CODEX/03_SECTIONS/272_PLATFORM_ENGINEERING.txt

SHA-256 2e69bcd495dc6468d4047ea4357b4db7a845f7fc01440714e33a42e88646934e

5,563 BYTES / PRESERVED SOURCE IDENTITY

SECTION 272 — Platform Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Create and operate a governed internal platform that gives teams reliable, secure, observable, and self-service capabilities without hiding ownership or imposing unsafe defaults.

## REQUIRED OUTCOME

A platform model with users, products, paved paths, interfaces, guardrails, tenancy, identity, self-service, reliability, support, cost, change, feedback, and retirement.

## WHY THIS MATTERS

A platform can reduce cognitive load and improve consistency, but an opaque or overcentralized platform becomes a bottleneck, spreads failure, and removes local understanding or control.

## APPLICABILITY

Apply when a team provides reusable infrastructure, tooling, workflows, environments, services, templates, or developer-facing capabilities to other teams or users.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no shared platform, self-service product, reusable path, or team dependency changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use enterprise, solution, infrastructure, cloud, security, identity, data, DevOps, operations, support, cost, and product baselines. Identify platform owner, consumers, service boundaries, and feedback authority.

## REQUIRED INPUTS

Consumer needs, platform capabilities, interfaces, templates, environments, identity, policy, reliability, cost, support, usage, incidents, roadmap, and retirement constraints.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Platform owners define service commitments, guardrails, support, access, cost, change, exceptions, and retirement. Consumer teams retain responsibility for their workloads and can challenge or exit a platform path.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized platform telemetry and feedback, draft templates and documentation, and propose improvements. It must not provision resources, grant access, change guardrails, hide consumer impact, or claim self-service safety without human authority.

## APPLICABLE EXISTING AGENT ROLES

Platform Engineer, Architect, Product Manager, User Experience Reviewer, Security Auditor, Operations Engineer, Implementer, Tester, Verifier, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Research consumers; define platform product and boundaries; design paved paths and escape hatches; secure and observe; provision; document; support; measure reliability, cost, adoption, and workload; manage change; and retire or migrate.

## HUMAN-AI INTERACTION REQUIREMENTS

Show consumer, capability, identity, guardrail, environment, cost, reliability, evidence, and limitation. Require human approval for access, policy, platform-wide change, destructive action, and service commitment.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use stable APIs, versioned templates, tenant isolation, least privilege, self-service with policy, observability, support, rollback, migration, portability, and independent emergency control. Avoid hidden coupling and single control-plane failure.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show platform health, consumers, versions, dependencies, cost, incidents, evidence freshness, support, owner, and authorized provision, restrict, migrate, rollback, override, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Platform charter and product catalog; consumer and service map; paved paths and interfaces; guardrail and access model; reliability and support; cost, change, migration, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify platform capability, version, consumer, environment, resource, policy, method, time, result, cost, reliability, and limitation. Adoption or provisioning count proves neither usefulness nor safety.

## VERIFICATION AND ACCEPTANCE TESTS

Verify self-service, isolation, permissions, guardrails, reliability, cost, observability, support, migration, rollback, accessibility, consumer outcome, and platform failure. Test escape hatch and provider loss.

## FAILURE STATES AND ANTI-PATTERNS

Platform as gatekeeper, hidden workload transfer, overbroad permissions, template drift, no escape hatch, adoption metric theater, central outage, and consumer responsibility made ambiguous.

## RECOVERY AND RECONSIDERATION

Pause unsafe platform capability, protect consumers, restore or roll back, communicate, provide manual or alternate path, and reassess workload and contract impact. Reopen on consumer, platform, provider, policy, or architecture change.

## COMPLETION CRITERIA

Consumers, capabilities, boundaries, interfaces, guardrails, ownership, support, cost, reliability, evidence, recovery, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner and consumer acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve platform versions, consumer map, contracts, templates, incidents, decisions, evidence, exceptions, migrations, and open gaps. Hand off to DevOps, DevSecOps, GitOps, SRE, Security, Support, Product, and Governance.

## RELATED CODEX SECTIONS

021-028 Enterprise through Functional Architecture; 263 Environment Management; 269-271 DevOps, DevSecOps, and GitOps; 301-330 Operations; 373 Roadmap Management; 389 Deprecation Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, architecture, platform, delivery, operations, and owner-control context, and the integration roadmap. Exact consumers, platform scope, service levels, cost, guardrails, support, portability, and acceptance authority remain unresolved.

# 273

## SECTION 273 - DEVELOPER EXPERIENCE

CODEX/03_SECTIONS/273_DEVELOPER_EXPERIENCE.txt

SHA-256 d542cfaa632befb971ff9274de840fea5de9fef00584c0bbd038ab4728747feb

5,064 BYTES / PRESERVED SOURCE IDENTITY

SECTION 273 — Developer Experience

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design the tools, workflows, documentation, environments, feedback, and support that let developers and technical contributors perform authorized work effectively, safely, accessibly, and sustainably.

## REQUIRED OUTCOME

A developer-experience model with user goals, journeys, setup, tooling, access, feedback, failure recovery, security, accessibility, support, measures, ownership, and retirement.

## WHY THIS MATTERS

Poor developer experience creates hidden work, unsafe workarounds, onboarding delay, inconsistent environments, and operational burden. Speed metrics alone can reward shortcuts that increase defects and fatigue.

## APPLICABILITY

Apply when contributors use a shared development workflow, platform, toolchain, repository, environment, documentation, or support path.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no contributor journey, tool, access, workflow, support, or quality outcome changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use user research, architecture, source, platform, security, identity, accessibility, quality, DevOps, support, and governance baselines. Identify contributor populations and product owner.

## REQUIRED INPUTS

Contributor goals, journeys, tools, repositories, environments, access, documentation, onboarding, support, telemetry, friction, incidents, accessibility, and outcomes.

## HUMAN RESPONSIBILITIES AND AUTHORITY

DX owners prioritize contributor outcomes and accessibility; teams own truthful documentation and support; humans approve access, policy, tool changes, and workload tradeoffs.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may summarize authorized feedback, draft docs and setup, identify friction, and propose improvements. It must not infer user intent, expose source or credentials, grant access, or change shared workflows without authority.

## APPLICABLE EXISTING AGENT ROLES

User Experience Reviewer, Researcher, Developer Experience Engineer, Platform Engineer, Documentation Writer, Security Auditor, Analyst, Tester, Verifier, and Critic.

## REQUIRED ACTIVITIES

Research contributors; map journeys; simplify setup; secure access; provide examples and diagnostics; test tools and documentation; measure friction and outcomes; support recovery; review changes; and retire obsolete paths.

## HUMAN-AI INTERACTION REQUIREMENTS

Show contributor, journey, tool, version, access, evidence, uncertainty, and impact. Require human review for shared guidance, access, security, and policy changes. Preserve accessible and manual support.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Provide reproducible setup, self-service with guardrails, clear errors, diagnostics, documentation versioning, isolated environments, secure defaults, accessibility, and recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show journey health, setup failures, tool versions, support demand, evidence freshness, owner, and authorized prioritize, change, rollback, restrict, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

DX charter; contributor and journey map; tool and environment catalog; setup and support paths; feedback and measure model; test, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify contributor, journey, tool, version, method, period, source, result, and limitation. Onboarding time or usage count proves only its measured slice.

## VERIFICATION AND ACCEPTANCE TESTS

Verify setup, access, documentation, tooling, accessibility, security, diagnostics, recovery, support, and representative contributor outcomes. Test clean machine, failure, and role variation.

## FAILURE STATES AND ANTI-PATTERNS

Tool-first design, tribal knowledge, setup requiring hidden access, documentation drift, metrics without user evidence, unsafe workaround, and support burden shifted to individuals.

## RECOVERY AND RECONSIDERATION

Restore prior path, provide support, protect credentials, preserve feedback, and correct tooling or documentation. Reopen on contributor, platform, policy, security, or workflow change.

## COMPLETION CRITERIA

Users, journeys, tools, access, support, accessibility, evidence, outcomes, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending contributor and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve journeys, tool versions, docs, feedback, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Tooling, Platform, DevOps, Security, Support, Documentation, and Governance.

## RELATED CODEX SECTIONS

001 User Research; 048 Accessibility Architecture; 259 Source Control; 269-272 DevOps, DevSecOps, GitOps, and Platform; 341 Documentation Engineering; 402-404 Review.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, user, platform, delivery, and support context, and the integration roadmap. Exact contributors, tools, measures, access, accessibility, support, and acceptance authority remain unresolved.

# 274

## SECTION 274 - TOOLING ENGINEERING

CODEX/03_SECTIONS/274_TOOLING_ENGINEERING.txt

SHA-256 226ba0a2691c510abc907661ba075e79bbe084e18c48ee4bd76a80bb28e3c2c4

4,728 BYTES / PRESERVED SOURCE IDENTITY

SECTION 274 — Tooling Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Engineer tools that make development, testing, delivery, operations, analysis, and support more reliable, observable, secure, accessible, and maintainable.

## REQUIRED OUTCOME

A tooling model with users, purpose, interfaces, dependencies, permissions, data, versions, installation, support, telemetry, quality, lifecycle, and fallback.

## WHY THIS MATTERS

Tools become part of the system and workflow. Unmaintained or overprivileged tools can leak data, corrupt source, create hidden coupling, or make recovery dependent on one person or provider.

## APPLICABILITY

Apply when a custom or shared tool materially affects engineering, testing, delivery, operations, governance, or user control.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when the tool has no material user, access, data, automation, quality, security, or lifecycle effect and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use user, platform, source, package, security, identity, privacy, testing, documentation, support, and operations baselines.

## REQUIRED INPUTS

Tool purpose, users, workflow, interfaces, dependencies, permissions, data, version, installation, telemetry, support, failure, and retirement.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Tool owners approve purpose, access, data, release, support, exceptions, and retirement. Users report defects and unsafe behavior; humans decide shared adoption and risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft tool code, docs, tests, and diagnostics from authorized context. It must not execute privileged actions, access secrets, alter shared tools, or infer user authorization without explicit control.

## APPLICABLE EXISTING AGENT ROLES

Tooling Engineer, Architect, Developer Experience Engineer, Implementer, Tester, Verifier, Security Auditor, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define users and purpose; design interfaces and access; implement; test behavior, security, accessibility, and recovery; package; document; support; observe; update; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show tool, version, user, permission, data, action, evidence, and limitation. Require human review for privileged tool behavior, shared release, data handling, and destructive operations.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use least privilege, clear APIs, isolation, reproducible install, versioning, safe defaults, audit, offline or fallback path, and dependency transparency.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show tool health, version, users, permissions, dependencies, incidents, evidence freshness, owner, and authorized upgrade, restrict, rollback, disable, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Tool charter; user and workflow model; interface and access design; build and test; documentation; support, incident, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify tool, version, user, environment, method, source, result, time, actor, and limitation. Installation or successful invocation proves neither safety nor usefulness.

## VERIFICATION AND ACCEPTANCE TESTS

Verify install, permissions, data, functionality, accessibility, security, upgrade, failure, recovery, support, compatibility, and rollback. Test least-privilege and degraded paths.

## FAILURE STATES AND ANTI-PATTERNS

Overprivileged tool, undocumented behavior, hidden network access, stale dependency, no fallback, personal ownership, and tool usage metric treated as outcome.

## RECOVERY AND RECONSIDERATION

Disable or roll back, protect data and credentials, preserve evidence, provide alternate path, and reassess users and workflows. Reopen on tool, dependency, platform, or risk change.

## COMPLETION CRITERIA

Purpose, users, permissions, data, version, dependencies, tests, support, evidence, recovery, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve source, packages, versions, docs, tests, incidents, decisions, evidence, exceptions, and open gaps. Hand off to DX, Platform, Build, Security, Testing, Support, and Operations.

## RELATED CODEX SECTIONS

259-265 Source, Dependency, Package, Artifact, Build, and Automation; 273 Developer Experience; 272 Platform Engineering; 341 Documentation Engineering; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, tooling, platform, security, and user context, and the integration roadmap. Exact tools, users, data, permissions, support, providers, and acceptance authority remain unresolved.

# 275

## SECTION 275 - INFRASTRUCTURE ENGINEERING

CODEX/03_SECTIONS/275_INFRASTRUCTURE_ENGINEERING.txt

SHA-256 dda0706ad5d1a4adc3e457acea029d2143a7cfd26378d296767b6f7cf8498eab

5,039 BYTES / PRESERVED SOURCE IDENTITY

SECTION 275 — Infrastructure Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design, provision, secure, operate, observe, scale, recover, and retire computing, network, storage, facility, and service infrastructure that supports declared outcomes.

## REQUIRED OUTCOME

An infrastructure model with resources, dependencies, topology, identity, configuration, capacity, availability, security, cost, observability, recovery, and lifecycle ownership.

## WHY THIS MATTERS

Infrastructure failures can affect many services at once. Manual state, common failure domains, insecure defaults, provider dependence, and undocumented capacity make recovery slow and evidence weak.

## APPLICABILITY

Apply when physical, virtual, cloud, edge, network, storage, database, facility, or shared platform resources affect a system or service.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no infrastructure resource, dependency, configuration, capacity, security, or operational behavior changes and an approved exact baseline applies.

## PREREQUISITES AND DEPENDENCIES

Use architecture, identity, security, configuration, environment, IaC, cloud, network, data, capacity, continuity, and operations baselines.

## REQUIRED INPUTS

Outcome, topology, resources, environments, identity, configuration, dependencies, workload, capacity, availability, security, cost, telemetry, recovery, and retirement.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Infrastructure owners approve design, access, capacity, cost, maintenance, change, exceptions, outage, recovery, and retirement. Humans accept service and provider risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized topology, telemetry, plans, and incidents; draft infrastructure code and tests; and propose optimization. It must not provision, modify, scale, destroy, or expose infrastructure without authority.

## APPLICABLE EXISTING AGENT ROLES

Infrastructure Engineer, Architect, Platform Engineer, Security Auditor, Network Engineer, Database Administrator, Storage Administrator, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Define service and topology; select resources; secure identity and configuration; provision; validate; observe; capacity-plan; maintain; back up; recover; optimize; document; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show resource, environment, owner, identity, change, dependency, cost, evidence, and limitation. Require human approval for production, privileged, destructive, security, cost, and provider changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use IaC, segmentation, redundancy, isolation, least privilege, immutable or versioned resources, observability, backup, restore, failover, and provider-independent recovery where warranted.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show topology, resource health, drift, capacity, cost, incidents, evidence freshness, owner, and authorized provision, scale, isolate, restore, rollback, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Infrastructure charter and topology; resource and dependency catalog; IaC and configuration; capacity and cost; security and observability; recovery, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify resource, version, environment, configuration, provider, method, time, actor, result, cost, and limitation. A plan or metric proves intended or observed state only.

## VERIFICATION AND ACCEPTANCE TESTS

Verify provisioning, identity, segmentation, configuration, capacity, availability, security, cost, monitoring, backup, restore, failover, rollback, and retirement.

## FAILURE STATES AND ANTI-PATTERNS

Manual drift, shared credentials, common failure domain ignored, capacity by guess, provider lock-in without plan, no recovery path, and infrastructure metric treated as service outcome.

## RECOVERY AND RECONSIDERATION

Isolate, preserve state, restore or rebuild, fail over, rotate credentials, communicate, and reassess service and data impact. Reopen on topology, provider, workload, dependency, or risk change.

## COMPLETION CRITERIA

Resources, topology, owners, configuration, identity, capacity, cost, security, evidence, recovery, lifecycle, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve topology, IaC, state, changes, incidents, backups, recovery, decisions, evidence, exceptions, and open gaps. Hand off to Cloud, Edge, Network, Database, Storage, Platform, Security, and Operations.

## RELATED CODEX SECTIONS

029 Operational Architecture; 038-044 Infrastructure, Cloud, Edge, Distributed, Network, Communications, and Integration Architecture; 258-272 Delivery; 301-330 Operations.

CODEX/03_SECTIONS/275_INFRASTRUCTURE_ENGINEERING.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, infrastructure, security, delivery, and operations context, and the integration roadmap. Exact resources, providers, topology, capacity, recovery, cost, and acceptance authority remain unresolved.

# 276

## SECTION 276 - INFRASTRUCTURE AS CODE

CODEX/03_SECTIONS/276_INFRASTRUCTURE_AS_CODE.txt

SHA-256 a45f05a198f25320465b65089b427b02edfaeec6e57a48a7ad6f227bace97cab

4,778 BYTES / PRESERVED SOURCE IDENTITY

SECTION 276 — Infrastructure as Code

## STATUS: CONTENT-COMPLETE

## PURPOSE

Express infrastructure intent as versioned, reviewed, testable, repeatable, and auditable code or declarative configuration with controlled state and recovery.

## REQUIRED OUTCOME

An IaC model with modules, inputs, state, providers, secrets boundary, policy, plan, approval, apply, drift, rollback, import, destruction, and evidence.

## WHY THIS MATTERS

IaC improves repeatability but can encode destructive or overprivileged behavior and can diverge from actual state. A plan is not an apply, and a successful apply is not proof of safe operation.

## APPLICABILITY

Apply when code or declarative configuration provisions, changes, observes, or retires infrastructure, platform, network, storage, identity, or environment resources.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no infrastructure state is managed declaratively or no IaC behavior changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use infrastructure, configuration, source, identity, policy, secrets, provider, environment, security, testing, and recovery baselines.

## REQUIRED INPUTS

Modules, inputs, state, provider, resources, dependencies, policy, plan, environment, secrets references, approvals, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve modules, providers, state, plan, apply, import, destroy, exception, and recovery. Humans review blast radius and irreversible changes.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft IaC, analyze plans and drift, and propose validation. It must not apply, destroy, access state secrets, change provider or policy, or represent a plan as actual state without authority.

## APPLICABLE EXISTING AGENT ROLES

IaC Engineer, Infrastructure Engineer, Architect, Platform Engineer, Security Auditor, Tester, Verifier, Implementer, Analyst, Critic, and Documentation Writer.

## REQUIRED ACTIVITIES

Define modules and inputs; secure state; validate and lint; plan; review; policy-check; apply; observe; detect drift; import or reconcile; recover; and retire resources and modules.

## HUMAN-AI INTERACTION REQUIREMENTS

Show module, input, provider, target, plan, state, identity, impact, evidence, and limitation. Require human approval for apply, destroy, import, secret, provider, and production action.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use modularity, pinned providers, remote state protection, policy gates, least privilege, plan/apply separation, drift detection, locking, idempotence, rollback, and state recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show plan, target, diff, state, drift, owner, provider, evidence freshness, and authorized review, apply, cancel, rollback, import, quarantine, and destroy controls.

## REQUIRED OUTPUTS AND ARTIFACTS

IaC modules and policy; state and provider record; plan and approval; apply and drift; recovery and destruction; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify code ref, module, input, provider, target, state version, plan, actor, time, result, and limitation. Plan output proves intended diff, not actual successful or safe state.

## VERIFICATION AND ACCEPTANCE TESTS

Verify syntax, policy, permissions, state, plan, idempotence, drift, apply, rollback, import, destroy guard, provider failure, and recovery. Test malicious or unexpected input.

## FAILURE STATES AND ANTI-PATTERNS

Unreviewed destroy, state exposure, floating provider, plan ignored, drift overwritten, module overprivilege, no state backup, and apply treated as service acceptance.

## RECOVERY AND RECONSIDERATION

Stop apply, preserve state, restore or reconcile, rotate credentials, roll back or repair, and reassess dependencies and data. Reopen on module, provider, input, target, or policy change.

## COMPLETION CRITERIA

Modules, inputs, state, providers, policy, plans, approvals, drift, recovery, evidence, owners, limits, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve modules, state versions, plans, approvals, applies, drift, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Infrastructure, Platform, Configuration, Security, GitOps, and Operations.

## RELATED CODEX SECTIONS

258 Configuration; 263 Environments; 271 GitOps; 275 Infrastructure Engineering; 277-286 Infrastructure Layers; 301-330 Operations and Recovery.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, infrastructure, delivery, security, and operations context, and the integration roadmap. Exact tool, state, providers, modules, policy, recovery, and acceptance authority remain unresolved.

# 277

## SECTION 277 - CLOUD ENGINEERING

CODEX/03_SECTIONS/277_CLOUD_ENGINEERING.txt

SHA-256 4d0086f26537454f3936f3940e06524b8429fd5d144308ed18e8b185723c4110

5,140 BYTES / PRESERVED SOURCE IDENTITY

SECTION 277 — Cloud Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design and operate cloud resources, services, accounts, networks, identities, data, automation, cost, security, resilience, and portability for declared outcomes.

## REQUIRED OUTCOME

A cloud model with provider, account or tenant, region, resources, identity, network, data, service limits, availability, cost, observability, governance, recovery, and exit.

## WHY THIS MATTERS

Cloud speed and abstraction can hide shared failure, provider dependency, data residency, cost, identity, and recovery assumptions. A deployed resource is not automatically secure, resilient, or owned.

## APPLICABILITY

Apply when a public, private, hybrid, or managed cloud provides computing, storage, network, data, identity, platform, AI, or operations capability.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no cloud resource, provider, region, account, data, identity, cost, or operational behavior changes and an approved exact baseline applies.

## PREREQUISITES AND DEPENDENCIES

Use enterprise, cloud, infrastructure, identity, security, privacy, data, cost, resilience, backup, exit, and operations baselines.

## REQUIRED INPUTS

Provider, account, region, resource, topology, identity, data, workload, service limits, SLA, cost, telemetry, compliance, backup, restore, and exit requirements.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Cloud owners approve provider, region, data, identity, cost, architecture, service limits, exceptions, outage, recovery, and exit. Humans accept provider and shared-responsibility risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized cloud metadata, plans, metrics, and cost; draft IaC and policies; and propose optimization. It must not access tenants, provision, change policy, delete resources, or claim cloud security without authority.

## APPLICABLE EXISTING AGENT ROLES

Cloud Engineer, Architect, Infrastructure Engineer, Security Auditor, FinOps Analyst, Data Engineer, Platform Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Define provider and boundaries; design accounts, network, identity, resources, data, availability, cost, observability, backup, recovery, compliance, portability, and retirement; provision; validate; monitor; and improve.

## HUMAN-AI INTERACTION REQUIREMENTS

Show provider, account, region, resource, identity, data, cost, change, evidence, and limitation. Require human approval for privileged, data, regional, destructive, cost, and provider changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use landing zones, tenancy, segmentation, least privilege, encryption, policy, resource limits, multi-zone design, backups, provider abstractions where useful, and tested exit or recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show cloud topology, health, resource, cost, data region, identity, findings, evidence freshness, owner, and authorized scale, isolate, restore, restrict, migrate, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Cloud charter; account and region model; resource and dependency catalog; security, cost, observability, backup, recovery, exit, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify provider, account, region, resource, configuration, identity, data, method, time, result, cost, owner, and limitation. Provider certification or dashboard proves only its declared scope.

## VERIFICATION AND ACCEPTANCE TESTS

Verify identity, segmentation, encryption, policy, limits, availability, cost, telemetry, backup, restore, failover, regional behavior, portability, and deletion. Test provider and control-plane outage.

## FAILURE STATES AND ANTI-PATTERNS

Shared responsibility misunderstood, admin credentials everywhere, region or data ignored, uncontrolled cost, provider lock-in without exit, backup untested, and cloud health equated with service outcome.

## RECOVERY AND RECONSIDERATION

Restrict affected resources, preserve state and logs, fail over or restore, rotate credentials, communicate, and reassess data, cost, user, and provider impact. Reopen on provider, region, service, workload, or policy change.

## COMPLETION CRITERIA

Provider, boundaries, resources, identity, data, cost, security, resilience, evidence, recovery, exit, owners, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve cloud inventory, IaC, accounts, policies, metrics, cost, incidents, backups, decisions, evidence, exceptions, and open gaps. Hand off to Infrastructure, Platform, Security, FinOps, Data, and Operations.

## RELATED CODEX SECTIONS

275-276 Infrastructure and IaC; 278-283 Edge, Containers, Virtualization, Servers, and Administration; 301-330 Operations and Recovery; 391-400 Data and Retirement.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, cloud, infrastructure, security, data, cost, and operations context, and the integration roadmap. Exact provider, regions, services, contracts, cost, exit, recovery, and acceptance authority remain unresolved.

# 278

## SECTION 278 - EDGE COMPUTING ENGINEERING

CODEX/03_SECTIONS/278_EDGE_COMPUTING_ENGINEERING.txt

SHA-256 6d96d83916d07edab3f91a97090448ba0508565974195476fd22818ab476a308

5,189 BYTES / PRESERVED SOURCE IDENTITY

SECTION 278 — Edge Computing Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design and operate computation, storage, networking, identity, data, and control close to users, devices, or physical environments under limited connectivity and distributed authority.

## REQUIRED OUTCOME

An edge model with site and node inventory, autonomy, synchronization, security, update, telemetry, resource limits, safety, offline behavior, recovery, and fleet ownership.

## WHY THIS MATTERS

Edge nodes operate in variable physical and network conditions with partial visibility and local consequences. A central control plane cannot be assumed available when an edge action or recovery decision is needed.

## APPLICABILITY

Apply when computation or data processing occurs outside a central facility or cloud, including devices, sites, gateways, field systems, and distributed nodes.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no edge node, site, offline behavior, synchronization, physical environment, or distributed control changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use infrastructure, embedded, cloud, network, identity, data, security, safety, fleet, operations, and recovery baselines.

## REQUIRED INPUTS

Nodes, sites, hardware, software, data, identity, connectivity, autonomy, synchronization, updates, telemetry, resource, safety, recovery, and disposal.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve autonomy, site access, update, data, safety, isolation, recovery, and retirement. Field operators have clear local stop and manual control; humans accept fleet and site risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized fleet metadata, telemetry, drift, and incidents; draft rollout and recovery plans; and propose local optimization. It must not issue physical or consequential edge commands, access private site data, or update a fleet without authority.

## APPLICABLE EXISTING AGENT ROLES

Edge Engineer, Infrastructure Engineer, Embedded Engineer, Network Engineer, Security Auditor, Fleet Operator, Tester, Verifier, Safety Reviewer, and Operations Engineer.

## REQUIRED ACTIVITIES

Inventory sites and nodes; define autonomy and trust; secure provisioning; design synchronization and offline; manage resources and updates; observe; test failure and safety; recover; replace; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show site, node, version, connectivity, data, identity, command, authority, evidence, and limitation. Require human approval for physical, safety, fleet-wide, data, update, and destruction action.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Support local safe state, bounded autonomy, eventual consistency, conflict resolution, offline credentials, fleet identity, staged update, telemetry buffering, tamper, and independent recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show fleet and node posture, site, connectivity, version, data, safety, evidence freshness, owner, and authorized isolate, update, pause, recover, replace, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Edge charter; site and node inventory; autonomy and synchronization design; update and telemetry; security and safety; fleet recovery, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify site, node, version, hardware, connectivity, data, identity, command, method, time, result, operator, and limitation. Central health proves neither local state nor offline safety.

## VERIFICATION AND ACCEPTANCE TESTS

Verify offline, network change, synchronization conflict, update, rollback, identity, local safety, resource, telemetry, recovery, physical access, and fleet isolation. Test control-plane loss.

## FAILURE STATES AND ANTI-PATTERNS

Central dependency assumed, fleet-wide update, stale commands, split brain, shared device identity, telemetry treated as complete, and remote recovery without local safety.

## RECOVERY AND RECONSIDERATION

Enter local safe mode, isolate, preserve evidence, restore known-good image, reconcile data, provide field support, and reassess site and fleet impact. Reopen on node, site, network, threat, hardware, or policy change.

## COMPLETION CRITERIA

Sites, nodes, autonomy, connectivity, data, identity, updates, safety, evidence, recovery, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending qualified review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve fleet identity, versions, commands, telemetry, incidents, updates, recovery, decisions, evidence, exceptions, and open gaps. Hand off to Cloud, Embedded, Network, Security, Field Operations, Safety, and Disposal.

## RELATED CODEX SECTIONS

058-060 Hardware, Embedded, and Firmware; 198-199 Embedded and Hardware Security; 277 Cloud; 284 Network; 298 Field Operations; 301-330 Operations; 379 Firmware Updates.

CODEX/03_SECTIONS/278_EDGE_COMPUTING_ENGINEERING.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, edge, infrastructure, security, safety, and operations context, and the integration roadmap. Exact sites, nodes, autonomy, connectivity, safety, data, fleet, recovery, and acceptance authority remain unresolved.

# 279

## SECTION 279 - CONTAINER ENGINEERING

CODEX/03_SECTIONS/279_CONTAINER_ENGINEERING.txt

SHA-256 53a9495efd0e2114cf8403b7fed102ab7886385f090293609aba31c65994cb26

4,769 BYTES / PRESERVED SOURCE IDENTITY

SECTION 279 — Container Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Build, secure, configure, distribute, run, observe, update, and retire container images and workloads with explicit isolation, identity, resource, and provenance.

## REQUIRED OUTCOME

A container model with image source, digest, build, dependencies, runtime identity, filesystem, network, resource limits, secrets, registry, vulnerability, lifecycle, and recovery.

## WHY THIS MATTERS

Containers package application and supply-chain behavior but do not automatically provide isolation, security, or reproducibility. Mutable tags, root privileges, host access, and image drift can create broad exposure.

## APPLICABILITY

Apply when containers package or run application, infrastructure, data, model, or tooling workloads.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no container image or runtime behavior changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use package, dependency, build, artifact, platform, identity, network, storage, security, secrets, and operations baselines.

## REQUIRED INPUTS

Image, digest, source, build, packages, runtime, identity, filesystem, network, resource, secrets, registry, policy, telemetry, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve image, registry, runtime privilege, resources, data, exceptions, deployment, and retirement. Humans accept isolation and host risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized image manifests, Dockerfiles or equivalent, scans, and runtime metadata; draft hardening and tests; and identify drift. It must not pull untrusted code into privileged context, change runtime, or promote images without authority.

## APPLICABLE EXISTING AGENT ROLES

Container Engineer, Security Auditor, Build Engineer, Platform Engineer, Dependency Manager, Tester, Verifier, Implementer, Analyst, and Documentation Writer.

## REQUIRED ACTIVITIES

Define image and runtime policy; build and scan; pin digest; minimize image; secure identity and filesystem; limit resources and network; run; observe; update; rollback; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show image, digest, source, registry, runtime identity, privilege, resource, network, evidence, and limitation. Require human approval for privileged, host, data, registry, and production actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use non-root, read-only, minimal images, signed provenance, resource and network policy, secret injection, immutable digest, isolation, health, and graceful shutdown.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show image, digest, workload, runtime, privilege, resources, findings, evidence freshness, owner, and authorized deploy, restrict, restart, roll back, quarantine, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Container charter; image and registry policy; build and scan; runtime and resource model; deployment and recovery; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify image, digest, source, build, registry, runtime, host, identity, method, time, result, and limitation. Tag or process status proves neither exact image nor isolation.

## VERIFICATION AND ACCEPTANCE TESTS

Verify image provenance, vulnerabilities, privilege, filesystem, network, secrets, resources, health, shutdown, update, rollback, isolation, and recovery.

## FAILURE STATES AND ANTI-PATTERNS

Mutable tag, root, host socket, broad network, secrets in image, unbounded resources, image scan without runtime test, and restart used as recovery without state analysis.

## RECOVERY AND RECONSIDERATION

Quarantine workload or image, preserve logs, roll back or replace, rotate credentials, restore state, and reassess host and downstream impact. Reopen on image, runtime, host, registry, or policy change.

## COMPLETION CRITERIA

Image, runtime, identity, privilege, resources, registry, security, evidence, recovery, owners, limits, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve images, digests, manifests, scans, runtime config, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Orchestration, Platform, Build, Security, Release, and Operations.

## RELATED CODEX SECTIONS

260-262 Dependency, Package, and Artifact; 264-268 Build and Delivery; 280 Container Orchestration; 301-330 Operations; 380-381 Patching.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, container, supply-chain, security, and operations context, and the integration roadmap. Exact runtime, registry, host, policy, data, isolation, and acceptance authority remain unresolved.

# 280

## SECTION 280 - CONTAINER ORCHESTRATION

CODEX/03_SECTIONS/280_CONTAINER_ORCHESTRATION.txt

SHA-256 19aee3842a743115739b9a62fd83c297cb943b2cb436acf832eda5f841d59b82

4,785 BYTES / PRESERVED SOURCE IDENTITY

SECTION 280 — Container Orchestration

## STATUS: CONTENT-COMPLETE

## PURPOSE

Schedule, connect, secure, scale, observe, update, and recover container workloads across nodes, services, tenants, and failure domains.

## REQUIRED OUTCOME

An orchestration model with cluster, control plane, nodes, workloads, identity, network, storage, scheduling, health, policy, upgrades, capacity, and recovery ownership.

## WHY THIS MATTERS

Orchestration concentrates control over many workloads. A control-plane compromise, bad policy, node failure, rollout, or scheduling assumption can affect broad service and data scope.

## APPLICABILITY

Apply when a control plane schedules or manages containerized workloads across nodes or environments.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no orchestrator, cluster, workload scheduling, policy, or control-plane behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use container, platform, network, identity, storage, configuration, security, capacity, resilience, release, and operations baselines.

## REQUIRED INPUTS

Cluster, nodes, workloads, policies, identities, images, network, storage, scheduling, health, capacity, upgrades, telemetry, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Cluster and workload owners approve access, policies, placement, upgrade, scale, disruption, recovery, and retirement. Humans retain emergency control over the control plane.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized cluster state and manifests, identify scheduling or resource issues, and draft policies and runbooks. It must not grant cluster access, change workloads, drain nodes, or delete resources without authority.

## APPLICABLE EXISTING AGENT ROLES

Orchestration Engineer, Container Engineer, Platform Engineer, Security Auditor, Network Engineer, Storage Administrator, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Define cluster and tenancy; secure control plane; manage nodes and workload policy; schedule; observe; scale; roll out; upgrade; test disruption; recover; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show cluster, namespace or tenant, workload, identity, policy, node, action, evidence, and limitation. Require human approval for privileged, disruptive, production, and destructive actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use control-plane isolation, least privilege, network policy, resource quotas, disruption budgets, health checks, storage recovery, staged upgrades, node diversity, and audit.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show cluster health, workload, node, policy, capacity, rollout, incidents, evidence freshness, owner, and authorized cordon, drain, scale, pause, rollback, restore, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Cluster charter; control-plane and node model; workload and policy; capacity and upgrade plan; observability, disruption, recovery, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify cluster, node, workload, image, policy, identity, version, method, time, result, and limitation. Control-plane health proves neither application outcome nor data integrity.

## VERIFICATION AND ACCEPTANCE TESTS

Verify access, isolation, scheduling, resource, network, storage, health, rollout, upgrade, node loss, control-plane loss, backup, restore, and recovery.

## FAILURE STATES AND ANTI-PATTERNS

Cluster admin everywhere, namespace isolation assumed, unbounded resources, unsafe drain, health check theater, control-plane single point, and upgrade without rollback.

## RECOVERY AND RECONSIDERATION

Restrict control plane, preserve state, isolate workloads, restore or fail over, repair nodes, and reassess data and service impact. Reopen on cluster, policy, workload, provider, or version change.

## COMPLETION CRITERIA

Cluster, nodes, workloads, policy, identity, capacity, updates, evidence, recovery, owners, limits, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve cluster state, manifests, policies, versions, incidents, upgrades, decisions, evidence, exceptions, and open gaps. Hand off to Containers, Platform, Network, Storage, Security, Release, and Operations.

## RELATED CODEX SECTIONS

279 Container Engineering; 272 Platform Engineering; 277 Cloud; 284 Network; 286 Storage; 301-330 Operations and Recovery; 380-381 Patching.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, containers, platform, infrastructure, security, and operations context, and the integration roadmap. Exact orchestrator, cluster, tenancy, policy, recovery, provider, and acceptance authority remain unresolved.

# 281

## SECTION 281 - VIRTUALIZATION ENGINEERING

CODEX/03_SECTIONS/281_VIRTUALIZATION_ENGINEERING.txt

SHA-256 7db58b315248504d2a568f1a6c663415926b852e94672b13cd54b7eae401fa8f

4,703 BYTES / PRESERVED SOURCE IDENTITY

SECTION 281 — Virtualization Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design and operate virtual compute, networks, storage, images, hypervisors, tenancy, isolation, performance, backup, and recovery.

## REQUIRED OUTCOME

A virtualization model with host, guest, hypervisor, image, identity, resource, network, storage, tenancy, migration, monitoring, security, and lifecycle ownership.

## WHY THIS MATTERS

Virtualization increases utilization and flexibility while concentrating risk. Host compromise, noisy neighbors, image drift, resource overcommit, or shared management can cross guest boundaries.

## APPLICABILITY

Apply when virtual machines, hypervisors, images, virtual networks, or virtual storage support a workload or service.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no virtual resource, host, image, tenancy, migration, or management behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use infrastructure, cloud, server, storage, network, identity, image, security, capacity, backup, recovery, and operations baselines.

## REQUIRED INPUTS

Hosts, hypervisor, guests, images, resources, identity, network, storage, tenancy, migration, telemetry, backup, recovery, and retirement.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve host and guest access, overcommit, migration, image, backup, maintenance, recovery, exceptions, and retirement. Humans accept shared-failure risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized inventory, metrics, images, and plans; draft policy and tests; and propose optimization. It must not access hypervisor control, migrate, resize, or delete guests without authority.

## APPLICABLE EXISTING AGENT ROLES

Virtualization Engineer, Infrastructure Engineer, Security Auditor, Server Engineer, Storage Administrator, Network Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Inventory hosts and guests; secure management; define isolation and resource; manage images; provision; monitor; migrate; patch; back up; recover; test failure; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show host, guest, image, identity, resource, tenancy, action, evidence, and limitation. Require human approval for privileged, migration, data, host, and destructive action.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use management isolation, least privilege, image provenance, resource quotas, network and storage boundaries, live migration controls, host diversity, snapshot policy, and recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show host and guest health, resource, image, tenancy, incidents, evidence freshness, owner, and authorized migrate, resize, isolate, restore, roll back, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Virtualization charter; host and guest catalog; image and resource model; isolation and migration; backup and recovery; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify host, guest, hypervisor, image, resource, tenant, method, time, result, actor, and limitation. Host metric or snapshot proves only its captured state.

## VERIFICATION AND ACCEPTANCE TESTS

Verify management access, isolation, image, resource, network, storage, migration, backup, restore, host loss, guest failure, and recovery.

## FAILURE STATES AND ANTI-PATTERNS

Shared admin, guest isolation assumed, image without provenance, overcommit without evidence, snapshots as backup, host failure domain ignored, and migration without rollback.

## RECOVERY AND RECONSIDERATION

Isolate host or guest, preserve state, restore or migrate, rotate credentials, reconcile data, and reassess shared impact. Reopen on host, hypervisor, image, workload, or provider change.

## COMPLETION CRITERIA

Hosts, guests, images, resources, isolation, ownership, evidence, recovery, migration, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve host and guest inventory, images, configurations, migrations, backups, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Servers, Storage, Network, Security, Cloud, Platform, and Operations.

## RELATED CODEX SECTIONS

275 Infrastructure; 277 Cloud; 279-280 Containers and Orchestration; 282 Server Engineering; 284 Network; 286 Storage; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, virtualization, infrastructure, security, and operations context, and the integration roadmap. Exact hypervisor, hosts, guests, tenancy, resource, recovery, and acceptance authority remain unresolved.

# 282

## SECTION 282 - SERVER ENGINEERING

CODEX/03_SECTIONS/282_SERVER_ENGINEERING.txt

SHA-256 feb7670cd6508e384256574fc36b2c90da9659397563b025c66a8e8fccb5eab1

4,907 BYTES / PRESERVED SOURCE IDENTITY

SECTION 282 — Server Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design and operate physical or virtual servers with secure identity, operating system, resources, storage, network, patching, observability, availability, and recovery.

## REQUIRED OUTCOME

A server model with hardware or virtual host, operating system, services, identity, configuration, resource, network, storage, patch, monitoring, backup, and retirement ownership.

## WHY THIS MATTERS

Servers combine privileged software, data, network, and physical or virtual dependencies. Unpatched, overprivileged, poorly monitored, or undocumented servers create broad and persistent risk.

## APPLICABILITY

Apply when a server hosts services, data, agents, workloads, control planes, or infrastructure that affects an outcome.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no server, operating system, hosted service, configuration, resource, or lifecycle behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use infrastructure, virtualization, cloud, identity, configuration, patching, network, storage, security, backup, and operations baselines.

## REQUIRED INPUTS

Server, host, OS, services, identity, configuration, packages, network, storage, resources, telemetry, patch, backup, recovery, and retirement.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Server owners approve access, services, patching, maintenance, resource, backup, outage, recovery, exceptions, and retirement. Humans accept availability and security tradeoffs.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized server inventory, configuration, logs, and metrics; draft hardening and maintenance; and propose diagnosis. It must not access privileged state, change services, patch, reboot, or destroy without authority.

## APPLICABLE EXISTING AGENT ROLES

Server Engineer, Systems Administrator, Infrastructure Engineer, Security Auditor, Network Engineer, Storage Administrator, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Inventory; secure identity and OS; configure; install services; patch; monitor; capacity-plan; back up; test restore; maintain; troubleshoot; replace; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show server, host, OS, service, identity, change, impact, evidence, and limitation. Require human approval for privileged, patch, reboot, data, production, and destructive actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use least privilege, hardened image, segmentation, configuration baseline, service isolation, resource limits, logging, patch rollback, backup, restore, and out-of-band recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show server health, services, resources, patches, findings, storage, network, evidence freshness, owner, and authorized isolate, patch, reboot, restore, rollback, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Server catalog; OS and service baseline; access and configuration; patch and maintenance; telemetry, backup, recovery, incident, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify server, host, OS, service, version, configuration, method, time, actor, result, and limitation. A ping or uptime metric proves only reachability or observed availability.

## VERIFICATION AND ACCEPTANCE TESTS

Verify identity, hardening, services, patch, configuration, resources, network, storage, logging, backup, restore, failover, and retirement. Test unauthorized service and OS failure.

## FAILURE STATES AND ANTI-PATTERNS

Shared admin, unmanaged server, unsupported OS, patch without rollback, logs missing, backup untested, service dependency unknown, and uptime treated as correctness.

## RECOVERY AND RECONSIDERATION

Isolate, preserve logs and state, restore or rebuild, rotate credentials, fail over, communicate, and reassess service and data impact. Reopen on OS, service, host, dependency, or threat change.

## COMPLETION CRITERIA

Server, OS, services, identity, configuration, resources, patches, evidence, backup, recovery, owners, limits, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve server identity, images, configurations, changes, patches, incidents, backups, decisions, evidence, exceptions, and open gaps. Hand off to Administration, Network, Database, Storage, Security, Platform, and Operations.

## RELATED CODEX SECTIONS

275 Infrastructure; 281 Virtualization; 283 Systems Administration; 284 Network; 285 Database; 286 Storage; 378-381 Maintenance and Patching; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, infrastructure, systems, security, and operations context, and the integration roadmap. Exact server classes, OS, services, patch, recovery, users, and acceptance authority remain unresolved.

# 283

## SECTION 283 - SYSTEMS ADMINISTRATION

CODEX/03_SECTIONS/283_SYSTEMS_ADMINISTRATION.txt

SHA-256 35a20f299edd822a0607d7909248f6be2170c38197958de87247f6d1e3f538a6

4,896 BYTES / PRESERVED SOURCE IDENTITY

SECTION 283 — Systems Administration

## STATUS: CONTENT-COMPLETE

## PURPOSE

Operate and maintain systems, accounts, services, endpoints, configurations, jobs, users, and records so they remain secure, available, supportable, and recoverable.

## REQUIRED OUTCOME

A systems-administration model with inventory, ownership, access, routine work, change, patch, monitoring, support, incident, backup, recovery, and retirement procedures.

## WHY THIS MATTERS

Routine administration is where drift and privilege accumulate. Unrecorded commands, shared accounts, stale jobs, and workarounds can become permanent control failures.

## APPLICABILITY

Apply when people or automation administer servers, endpoints, services, accounts, configurations, jobs, or infrastructure.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no administrative action, account, service, configuration, or operational record changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use IAM, server, network, storage, database, configuration, patch, secrets, operations, incident, backup, and records baselines.

## REQUIRED INPUTS

Assets, accounts, services, jobs, runbooks, permissions, changes, patches, alerts, logs, support, backups, recovery, and retirement requirements.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Administrators operate within assigned privilege and record changes; owners approve access, maintenance, exceptions, emergency work, and retirement. Humans review high-impact and destructive actions.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized logs and inventory, draft runbooks, suggest diagnosis, and identify drift. It must not execute privileged commands, access secrets, disable controls, or alter systems without explicit delegated authority.

## APPLICABLE EXISTING AGENT ROLES

Systems Administrator, Operations Engineer, Security Auditor, Platform Engineer, Server Engineer, Network Engineer, Database Administrator, Storage Administrator, Tester, and Verifier.

## REQUIRED ACTIVITIES

Inventory and access; perform routine maintenance; change and patch; monitor; back up; troubleshoot; support users; respond to incidents; recover; review privilege; document; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show asset, account, command or action, privilege, target, evidence, impact, and limitation. Require human approval for privileged, destructive, production, data, and emergency actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use just-in-time privilege, session audit, runbooks, automation with guardrails, configuration management, out-of-band recovery, logging, and separation of administration from approval.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show administrative activity, assets, access, changes, alerts, evidence freshness, owner, and authorized approve, execute, revoke, isolate, recover, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Administration charter; asset and account catalog; runbooks; access and change records; patch and incident; backup and recovery; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify asset, administrator, account, privilege, action, command reference, time, method, result, and limitation. A command log proves activity, not authorization or correctness.

## VERIFICATION AND ACCEPTANCE TESTS

Verify access, logging, runbook, change, patch, backup, restore, alert, support, recovery, and retirement. Test least privilege, emergency, and failed automation.

## FAILURE STATES AND ANTI-PATTERNS

Shared admin, undocumented manual fix, privilege never reviewed, secrets in commands, alert ignored, routine job without owner, and recovery dependent on the same account or system.

## RECOVERY AND RECONSIDERATION

Stop unsafe action, preserve logs, revoke or rotate access, restore known-good state, communicate, and reassess downstream impact. Reopen on asset, admin, policy, tool, or environment change.

## COMPLETION CRITERIA

Assets, accounts, privileges, procedures, changes, evidence, recovery, owners, exceptions, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve runbooks, inventories, sessions, changes, incidents, backups, decisions, evidence, exceptions, and open gaps. Hand off to Server, Network, Database, Storage, Security, Platform, Support, and Operations.

## RELATED CODEX SECTIONS

200-205 IAM, Authentication, Authorization, Keys, and Secrets; 258 Configuration; 282 Servers; 284-286 Network, Database, and Storage; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, systems, security, administration, and operations context, and the integration roadmap. Exact assets, accounts, tools, privilege, runbooks, staffing, and acceptance authority remain unresolved.

# 284

## SECTION 284 - NETWORK ENGINEERING

CODEX/03_SECTIONS/284_NETWORK_ENGINEERING.txt

SHA-256 3bde12048eed4dda424b53da73bb90d43bdb0c5c946c2c083b751054457fd606

5,011 BYTES / PRESERVED SOURCE IDENTITY

SECTION 284 — Network Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Design, secure, operate, observe, scale, troubleshoot, and recover networks that connect people, devices, services, data, and control boundaries.

## REQUIRED OUTCOME

A network model with topology, addressing, identity, segmentation, routes, protocols, trust boundaries, performance, availability, monitoring, change, and recovery ownership.

## WHY THIS MATTERS

Networks carry authority and data across boundaries. Misrouting, flat trust, hidden dependencies, provider loss, and inaccurate telemetry can create broad exposure or make recovery impossible.

## APPLICABILITY

Apply when network, communication, routing, connectivity, name resolution, load balancing, firewall, wireless, or physical link affects a system or service.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no network boundary, route, protocol, connectivity, trust, or operational behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use architecture, identity, security, cloud, edge, server, database, storage, communications, configuration, monitoring, and operations baselines.

## REQUIRED INPUTS

Topology, assets, addresses, routes, protocols, identities, zones, policies, dependencies, capacity, latency, telemetry, provider, change, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Network owners approve topology, trust, routes, access, provider, maintenance, outage, exceptions, and recovery. Humans review changes crossing protected boundaries.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized topology, flow, metrics, configurations, and incidents; draft diagrams and tests; and propose diagnosis. It must not change routes, firewall, DNS, access, or capture content without authority.

## APPLICABLE EXISTING AGENT ROLES

Network Engineer, Security Auditor, Architect, Cloud Engineer, Systems Administrator, Observer, Tester, Verifier, Analyst, Critic, and Operations Engineer.

## REQUIRED ACTIVITIES

Define topology and zones; allocate addressing; secure identity and protocols; configure routing and policy; monitor; capacity-plan; test failure; maintain; troubleshoot; recover; and retire links and devices.

## HUMAN-AI INTERACTION REQUIREMENTS

Show topology, source, route, zone, identity, action, evidence, privacy boundary, and limitation. Require human approval for firewall, routing, capture, production, and destructive changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use segmentation, least privilege, authenticated management, resilient routing, provider diversity, rate and fault controls, observability, encryption, and out-of-band recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show topology, links, routes, zones, health, capacity, flows, incidents, evidence freshness, owner, and authorized isolate, reroute, restrict, restore, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Network charter and topology; addressing and trust model; routing and policy; capacity and monitoring; change, incident, recovery, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify link, device, route, zone, protocol, identity, configuration, method, time, result, actor, and limitation. Reachability proves neither authorization nor end-to-end service behavior.

## VERIFICATION AND ACCEPTANCE TESTS

Verify segmentation, routing, DNS, protocol, identity, encryption, capacity, latency, monitoring, failover, rollback, isolation, and recovery. Test provider, link, route, and control-plane loss.

## FAILURE STATES AND ANTI-PATTERNS

Flat network, implicit trust, shared admin, unmonitored route, firewall change without review, packet capture without privacy authority, and connectivity treated as security or service success.

## RECOVERY AND RECONSIDERATION

Isolate, preserve configuration and flow evidence, reroute or fail over, restore known-good policy, rotate credentials, communicate, and reassess data and user impact. Reopen on topology, provider, threat, or service change.

## COMPLETION CRITERIA

Topology, zones, routes, identities, policies, capacity, evidence, recovery, owners, limits, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve diagrams, configs, changes, flows, incidents, backups, decisions, evidence, exceptions, and open gaps. Hand off to Cloud, Edge, Servers, Database, Storage, Security, Communications, and Operations.

## RELATED CODEX SECTIONS

038-044 Network, Communications, Integration, API, Event, and Runtime Architecture; 277-278 Cloud and Edge; 282-286 Servers, Administration, Database, and Storage; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, network, security, infrastructure, and operations context, and the integration roadmap. Exact topology, providers, zones, protocols, capacity, privacy, recovery, and acceptance authority remain unresolved.

# 285

## SECTION 285 - DATABASE ADMINISTRATION

CODEX/03_SECTIONS/285_DATABASE_ADMINISTRATION.txt

SHA-256 f80fd3e4b4434edbbcabdfc6fed7a4046281680fb5b2a90ab617c3c93bb2984b

5,069 BYTES / PRESERVED SOURCE IDENTITY

SECTION 285 — Database Administration

## STATUS: CONTENT-COMPLETE

## PURPOSE

Operate and protect databases, schemas, users, queries, storage, replication, backup, recovery, performance, and lifecycle so data remains available, correct, secure, and governed.

## REQUIRED OUTCOME

A database-administration model with instances, schemas, owners, access, changes, capacity, performance, quality, backup, restore, replication, monitoring, retention, and retirement.

## WHY THIS MATTERS

Database actions can affect large populations and irreversible data state. Privileged access, unsafe migration, untested backup, replication lag, and hidden queries create operational and evidentiary risk.

## APPLICABILITY

Apply when a database or data service is provisioned, configured, queried, migrated, backed up, restored, replicated, secured, or retired.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no database, schema, access, data, query, performance, backup, or lifecycle behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use data, database, identity, security, privacy, configuration, migration, backup, recovery, performance, records, and operations baselines.

## REQUIRED INPUTS

Database, schema, data class, owner, users, roles, queries, migrations, indexes, replication, backup, restore, performance, retention, and deletion requirements.

## HUMAN RESPONSIBILITIES AND AUTHORITY

DB owners approve schema, access, migration, destructive query, backup, restore, replication, retention, and retirement. Humans review data meaning, privacy, and recovery impact.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized metadata, query plans, logs, and bounded samples; draft queries, tests, and runbooks; and propose tuning. It must not access raw sensitive data, execute writes or DDL, change access, or delete data without authority.

## APPLICABLE EXISTING AGENT ROLES

Database Administrator, Data Architect, Data Engineer, Security Auditor, Privacy Reviewer, Performance Engineer, Tester, Verifier, Analyst, and Documentation Writer.

## REQUIRED ACTIVITIES

Inventory and classify; secure access; manage schemas and migrations; monitor; tune; back up; test restore; replicate; handle incidents; reconcile data; retain and delete; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show database, schema, query, user, privilege, data scope, environment, action, evidence, and limitation. Require human approval for writes, DDL, privilege, migration, restore, deletion, and production tuning.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use least privilege, separation, transactions, migration rollback, replication, backups, encryption, audit, resource limits, query observability, and independent recovery path.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show database health, schema, access, queries, replication, storage, backup, performance, evidence freshness, owner, and authorized restrict, migrate, restore, fail over, correct, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Database catalog; schema and access model; migration and query record; performance and quality; backup, restore, incident, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify database, schema, query, user, data class, version, environment, method, time, result, actor, and limitation. Query success or row count proves neither semantic correctness nor data safety.

## VERIFICATION AND ACCEPTANCE TESTS

Verify access, schema, migration, transaction, query, backup, restore, replication, failover, performance, privacy, audit, retention, and deletion. Test unsafe query, lag, corruption, and provider loss.

## FAILURE STATES AND ANTI-PATTERNS

Shared admin, schema drift, migration without rollback, backup not restored, replication assumed current, sensitive data in logs, and performance tuning without workload or user context.

## RECOVERY AND RECONSIDERATION

Restrict writes, preserve logs and state, restore or fail over, reconcile, notify consumers, correct data, and reassess retention and user impact. Reopen on schema, data, workload, provider, or policy change.

## COMPLETION CRITERIA

Database, schemas, owners, access, changes, data, performance, backup, recovery, evidence, retention, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve schemas, migrations, access, queries, backups, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Data, Storage, Security, Application, Platform, Testing, Records, and Operations.

## RELATED CODEX SECTIONS

031 Data Architecture; 159-162 Warehouse, Lake, Analytics, and BI; 254 Data Testing; 286 Storage; 307 Data Operations; 391-395 Data Lifecycle.

CODEX/03_SECTIONS/285_DATABASE_ADMINISTRATION.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, data, database, security, and operations context, and the integration roadmap. Exact database, engine, schema, access, recovery, retention, workload, and acceptance authority remain unresolved.

# 286

## SECTION 286 - STORAGE ADMINISTRATION

CODEX/03_SECTIONS/286_STORAGE_ADMINISTRATION.txt

SHA-256 8737e1c7f9bd18d18bc20f317bbf0ebad8cbd39835f8519fc49709de00dba08d

5,122 BYTES / PRESERVED SOURCE IDENTITY

SECTION 286 — Storage Administration

## STATUS: CONTENT-COMPLETE

## PURPOSE

Operate and govern storage systems, volumes, files, objects, backups, replication, access, performance, retention, recovery, and disposal.

## REQUIRED OUTCOME

A storage model with classes, owners, location, capacity, performance, access, encryption, replication, backup, retention, deletion, monitoring, recovery, and lifecycle.

## WHY THIS MATTERS

Storage contains the durable state on which systems and people rely. Capacity exhaustion, access error, silent corruption, untested backup, retention drift, or unsafe deletion can create broad harm.

## APPLICABILITY

Apply when data, artifacts, backups, logs, evidence, databases, files, objects, or physical or virtual volumes are stored, moved, recovered, retained, or deleted.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no storage class, data, access, capacity, performance, backup, retention, or deletion behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use data, privacy, security, database, infrastructure, backup, archive, records, deletion, cost, and operations baselines.

## REQUIRED INPUTS

Storage, data class, owner, location, access, capacity, performance, encryption, replication, backup, retention, deletion, cost, telemetry, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Storage owners approve class, location, access, retention, deletion, backup, recovery, cost, and exceptions. Humans authorize restoration, legal hold, and destructive actions.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized metadata, capacity, performance, and lifecycle signals; draft plans; and identify anomalies. It must not read sensitive content, change access, delete data, migrate, or restore without authority.

## APPLICABLE EXISTING AGENT ROLES

Storage Administrator, Data Architect, Database Administrator, Security Auditor, Privacy Reviewer, Infrastructure Engineer, Tester, Verifier, Analyst, and Operations Engineer.

## REQUIRED ACTIVITIES

Classify and inventory; provision; secure access; encrypt; monitor capacity and performance; replicate and back up; test restore; retain and archive; delete; handle incident; and retire media.

## HUMAN-AI INTERACTION REQUIREMENTS

Show storage class, owner, location, data classification, access, action, evidence, and limitation without exposing content. Require human approval for access, restore, migration, retention, deletion, and disposal.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate operational, backup, archive, evidence, quarantine, and disposal states. Use immutable or protected backups, encryption, least privilege, integrity checks, capacity alerts, and recovery independent of the failed path.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show capacity, class, location, access, replication, backup, retention, cost, integrity, evidence freshness, owner, and authorized restrict, restore, migrate, delete, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Storage catalog and class model; access and protection; capacity and performance; backup and restore; retention and deletion; incident, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify storage, data class, owner, location, version, method, time, capacity, integrity, result, actor, and limitation. Object count or backup completion proves only its captured scope.

## VERIFICATION AND ACCEPTANCE TESTS

Verify access, encryption, integrity, capacity, performance, replication, backup, restore, deletion, retention, legal hold, cost, and recovery. Test corruption, full volume, provider loss, and accidental deletion.

## FAILURE STATES AND ANTI-PATTERNS

Backup as restore proof, unbounded retention, unrestricted bucket or share, capacity surprise, replica not deleted, storage tier assumed secure, and data content exposed during diagnosis.

## RECOVERY AND RECONSIDERATION

Restrict affected storage, preserve integrity, restore or fail over, reconcile, notify owners, correct access or retention, and reassess user and legal impact. Reopen on data, storage, provider, policy, or lifecycle change.

## COMPLETION CRITERIA

Storage, data, owners, access, protection, capacity, performance, backup, recovery, retention, deletion, evidence, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve storage inventory, classes, access, backups, restores, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Data, Database, Infrastructure, Security, Records, Archive, and Operations.

## RELATED CODEX SECTIONS

031 Data Architecture; 152 Storage Engineering; 225 Records; 285 Database Administration; 301-330 Operations and Recovery; 391-400 Data Lifecycle and Disposal.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, data, storage, privacy, security, and operations context, and the integration roadmap. Exact storage, classes, locations, providers, retention, recovery, cost, and acceptance authority remain unresolved.

# 287

## SECTION 287 - RELEASE ENGINEERING

CODEX/03_SECTIONS/287_RELEASE_ENGINEERING.txt

SHA-256 e265bf3d36364ce766e1da0d1b19dac4f6201154de14f87b25f93450eeb90adb

5,311 BYTES / PRESERVED SOURCE IDENTITY

SECTION 287 — Release Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Engineer repeatable, traceable, safe, and reversible release processes that turn accepted changes into communicated, supportable, and governed versions.

## REQUIRED OUTCOME

A release model with version, artifact, scope, readiness, notes, dependencies, approvals, packaging, communication, rollout, rollback, support, evidence, and retirement.

## WHY THIS MATTERS

A release is a social, technical, and operational event. A package can be correct while communication, support, migration, compatibility, or rollback is not ready.

## APPLICABILITY

Apply when a product, service, model, firmware, data, documentation, policy, infrastructure, or configuration version is made available to users, operators, or downstream systems.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no version, consumer, artifact, deployment, communication, support, or operational expectation changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use acceptance, delivery, deployment, artifact, configuration, compatibility, support, operations, security, legal, documentation, and rollback baselines.

## REQUIRED INPUTS

Version, artifact, changes, requirements, tests, findings, dependencies, compatibility, approvals, notes, migration, support, communication, rollout, rollback, and acceptance.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Release owners approve version, readiness, notes, support, communication, exceptions, timing, rollout, rollback, and retirement. Humans decide whether users and operators can safely receive the release.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may assemble change summaries, draft release notes and checklists, identify missing evidence, and propose rollout. It must not invent changes, disclose sensitive details, approve, publish, deploy, or communicate commitments without authority.

## APPLICABLE EXISTING AGENT ROLES

Release Engineer, Delivery Engineer, Product Manager, Documentation Writer, Tester, Verifier, Operations Engineer, Security Auditor, Support Reviewer, Coordinator, and Critic.

## REQUIRED ACTIVITIES

Baseline version and artifact; inspect changes; validate tests and dependencies; assess compatibility and risk; prepare notes, migration, support, and communication; approve; package and distribute; monitor; roll back or patch; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show version, artifact, audience, change, evidence, risk, dependency, communication, owner, and limitation. Require human approval for release, public notes, security or legal content, migration, and rollback.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use immutable version and artifact, signed provenance, staged distribution, compatibility and migration, feature flags, rollback, release registry, support hooks, and audit.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show release candidate, readiness, change, evidence freshness, audiences, support, incidents, rollout, rollback, and authorized approve, hold, publish, promote, roll back, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Release charter; version and artifact record; change and compatibility summary; readiness and approval; notes, migration, support, communication, rollout, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify version, artifact digest, source, tests, findings, approval, audience, environment, method, time, result, and limitation. Release publication proves availability, not successful adoption or operation.

## VERIFICATION AND ACCEPTANCE TESTS

Verify version and artifact, notes, compatibility, security, support, migration, approvals, distribution, rollback, monitoring, and retirement. Test partial, failed, and withdrawn release.

## FAILURE STATES AND ANTI-PATTERNS

Release from unverified source, notes that overclaim, hidden breaking change, support unprepared, migration without rollback, no withdrawal, and release count treated as product success.

## RECOVERY AND RECONSIDERATION

Hold or withdraw, preserve artifact and communications, roll back or patch, support affected users, reconcile data, and reassess commitments. Reopen on artifact, audience, dependency, incident, or compatibility change.

## COMPLETION CRITERIA

Version, artifact, changes, readiness, approvals, audiences, support, communication, rollout, rollback, evidence, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve release records, artifacts, approvals, notes, communications, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Delivery, Deployment, Operations, Support, Documentation, Security, and Product.

## RELATED CODEX SECTIONS

237 Acceptance Testing; 258-268 Configuration through Deployment; 273-274 Developer Experience and Tooling; 301-330 Operations; 373 Roadmap Management; 388-390 Version, Deprecation, and Sunset.

CODEX/03_SECTIONS/287_RELEASE_ENGINEERING.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, release, testing, support, operations, and owner-control context, and the integration roadmap. Exact release authority, versioning, audiences, compatibility, migration, communication, rollback, and acceptance evidence remain unresolved.

# 288

## SECTION 288 - RELEASE MANAGEMENT

CODEX/03_SECTIONS/288_RELEASE_MANAGEMENT.txt

SHA-256 134b5a6b5a5afab4aee84e4136ccf295fa337563e2806b6e62e9f24f78dd1145

4,909 BYTES / PRESERVED SOURCE IDENTITY

SECTION 288 — Release Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Coordinate the decision, readiness, timing, communication, risk, approvals, deployment, support, rollback, and follow-up for changes released to people or operations.

## REQUIRED OUTCOME

A release governance model with candidate, scope, readiness, decision rights, dependencies, communication, support, rollout, rollback, metrics, incident handling, and closure.

## WHY THIS MATTERS

Release is a cross-functional commitment, not a button. A technically complete change can fail through poor timing, unsupported users, unresolved risk, unprepared operations, or misleading communication.

## APPLICABILITY

Apply when a version, capability, policy, model, data, configuration, or infrastructure change is made available beyond its authoring context.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no consumer, environment, behavior, commitment, support, or operational risk changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use acceptance, testing, artifact, deployment, change, support, operations, security, privacy, communication, and rollback baselines.

## REQUIRED INPUTS

Release candidate, scope, artifact, tests, findings, dependencies, readiness, approvals, risk, support, communication, rollout, rollback, and owner decision.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Release authority approves readiness, timing, risk, communication, exception, rollout, rollback, and closure. Contributors provide truthful evidence; affected owners can hold the release.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may assemble readiness, draft notes, identify missing evidence, and summarize impacts. It must not approve, publish, deploy, communicate commitments, or close a release without accountable human authority.

## APPLICABLE EXISTING AGENT ROLES

Release Manager, Release Engineer, Product Manager, Tester, Verifier, Operations Engineer, Support Reviewer, Security Auditor, Coordinator, and Documentation Writer.

## REQUIRED ACTIVITIES

Define scope and audience; baseline candidate; review evidence and risks; confirm compatibility, migration, support, and communication; approve; roll out; monitor; rollback or pause; collect outcome; and close or reopen.

## HUMAN-AI INTERACTION REQUIREMENTS

Show candidate, scope, evidence, readiness, owner, risk, dependencies, support, communication, authority, and rollback. Require human approval for release and external commitments.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use release registry, immutable artifacts, staged rollout, feature flags, readiness gates, dependency tracking, rollback, support hooks, and post-release evidence.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show release state, approvals, readiness, dependencies, incidents, users, evidence freshness, support, rollout, rollback, and authorized approve, hold, publish, pause, rollback, and reopen controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Release charter; candidate and readiness record; risk and dependency assessment; communications and support; rollout and rollback; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify candidate, artifact, scope, environment, tests, approval, actor, time, result, impact, and limitation. Release approval proves a decision, not post-release success.

## VERIFICATION AND ACCEPTANCE TESTS

Verify evidence, approvals, audience, compatibility, support, communication, deployment, monitoring, rollback, migration, and closure. Exercise partial and failed release.

## FAILURE STATES AND ANTI-PATTERNS

Approval theater, hidden breaking change, release without support, notes that overclaim, no rollback, and release frequency treated as value.

## RECOVERY AND RECONSIDERATION

Hold or withdraw, preserve artifacts and decisions, roll back or patch, support users, communicate, and reassess impact. Reopen on incident, audience, artifact, or risk change.

## COMPLETION CRITERIA

Scope, candidate, evidence, approvals, support, communication, rollout, rollback, owner, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve release records, artifacts, approvals, notes, communications, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Deployment, Operations, Support, Product, Security, and Governance.

## RELATED CODEX SECTIONS

237 Acceptance Testing; 267-268 Delivery and Deployment; 287 Release Engineering; 289 Change Management; 301-330 Operations; 388-390 Version, Deprecation, and Sunset.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, release, testing, operations, and owner-control context, and the integration roadmap. Exact authority, readiness, audiences, communication, support, rollback, and acceptance evidence remain unresolved.

# 289

## SECTION 289 - CHANGE MANAGEMENT

CODEX/03_SECTIONS/289_CHANGE_MANAGEMENT.txt

SHA-256 9aebe55d562b083335ee425aad7b2f9f5ff21c5f8dc6758cc135f37d6ccda85f

4,920 BYTES / PRESERVED SOURCE IDENTITY

SECTION 289 — Change Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Control proposed, planned, emergency, rejected, implemented, failed, and retired change with proportionate impact analysis, authorization, evidence, and recovery.

## REQUIRED OUTCOME

A change model with request, scope, reason, impact, risk, dependencies, approval, implementation, validation, communication, rollback, closure, and learning.

## WHY THIS MATTERS

Change introduces uncertainty into a working system. Process can become either bureaucracy that teams bypass or a thin approval ritual that does not examine impact, evidence, or recovery.

## APPLICABILITY

Apply when a change can affect behavior, users, data, security, safety, cost, service, architecture, obligations, or continuity.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when impact analysis shows no material behavior, dependency, risk, or operating effect and an approved exact baseline covers the change class.

## PREREQUISITES AND DEPENDENCIES

Use configuration, source, release, deployment, testing, architecture, risk, security, operations, communication, and rollback baselines.

## REQUIRED INPUTS

Request, owner, reason, scope, baseline, impact, risk, dependencies, schedule, approvals, implementation, tests, communication, rollback, and closure evidence.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Change owners assess and implement; approvers decide risk, timing, exception, and rollback; operators can stop unsafe change. Emergency changes require later review, not absence of accountability.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may summarize diffs, map dependencies, draft impact and test plans, and identify missing evidence. It must not classify risk conclusively, approve, implement, bypass, or close a change without authority.

## APPLICABLE EXISTING AGENT ROLES

Change Manager, Architect, Planner, Implementer, Tester, Verifier, Operations Engineer, Security Auditor, Governance Reviewer, and Documentation Writer.

## REQUIRED ACTIVITIES

Capture request; baseline; assess impact and risk; plan test, communication, and rollback; approve; implement; observe; validate; close or recover; review emergency; and update dependencies and records.

## HUMAN-AI INTERACTION REQUIREMENTS

Show change, baseline, risk, owner, authority, dependencies, evidence, implementation state, and rollback. Require human approval for protected, production, data, security, safety, and emergency changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use traceable change records, dependency graph, policy gates, staged execution, immutable artifacts, feature flags, rollback, audit, and change-aware evidence invalidation.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show change state, impact, risk, approvals, dependencies, evidence freshness, implementation, incidents, rollback, and authorized approve, reject, schedule, pause, roll back, and close controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Change record; impact and risk assessment; implementation and test plan; approvals; communication; validation and rollback; emergency review; decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify change, baseline, owner, approver, scope, environment, method, time, result, impact, and limitation. Approval proves authorization, not successful or safe outcome.

## VERIFICATION AND ACCEPTANCE TESTS

Verify impact, approvals, tests, communication, implementation, monitoring, rollback, emergency review, closure, and traceability. Test failed, partial, conflicting, and out-of-window change.

## FAILURE STATES AND ANTI-PATTERNS

Approval without impact, change without baseline, emergency forever, hidden manual action, rollback untested, ticket closed by deployment, and process bypass caused by excessive friction.

## RECOVERY AND RECONSIDERATION

Pause or roll back, preserve state and evidence, communicate, recover, and reassess change and downstream impact. Reopen on incident, failed validation, new dependency, or invalidated evidence.

## COMPLETION CRITERIA

Request, scope, impact, risk, authority, implementation, validation, recovery, evidence, communication, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve change history, approvals, implementations, incidents, rollbacks, evidence, exceptions, and open gaps. Hand off to Release, Deployment, Configuration, Operations, Security, Testing, and Governance.

## RELATED CODEX SECTIONS

018 Risk Management; 258 Configuration; 267-268 Delivery and Deployment; 288 Release Management; 301-330 Operations; 386-390 Debt, Version, Deprecation, and Sunset.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, change, risk, release, and operations context, and the integration roadmap. Exact classes, authority, emergency, evidence, rollback, and acceptance rules remain unresolved.

# 290

## SECTION 290 - DEPLOYMENT ENGINEERING

CODEX/03_SECTIONS/290_DEPLOYMENT_ENGINEERING.txt

SHA-256 bb9fc86131bd7a8d1b45d97a5f640f2aebafa01e8123bf8ccdf1782959919c73

4,463 BYTES / PRESERVED SOURCE IDENTITY

SECTION 290 — Deployment Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Engineer repeatable, observable, secure, and reversible mechanisms for placing exact artifacts and configuration into declared targets.

## REQUIRED OUTCOME

A deployment mechanism with artifact, target, identity, strategy, configuration, migration, health, policy, rollback, audit, and recovery.

## WHY THIS MATTERS

Deployment changes live state. A mechanism that cannot identify what changed, stop safely, observe user impact, or recover from partial execution cannot support trustworthy operations.

## APPLICABILITY

Apply when software, firmware, infrastructure, configuration, data, models, or policies are placed or activated in an environment.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no target, artifact, configuration, data, or live state changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use artifact, environment, configuration, identity, testing, release, migration, backup, rollback, monitoring, and incident baselines.

## REQUIRED INPUTS

Artifact, target, version, identity, strategy, configuration, migration, health, gate, telemetry, rollback, and authorization.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Deployment owners approve target, strategy, gate, identity, migration, rollback, exception, and stop. Humans decide production and destructive deployment.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft manifests, analyze deployment state, and recommend pause or rollback. It must not deploy, widen target, change gates, or claim completion without authority.

## APPLICABLE EXISTING AGENT ROLES

Deployment Engineer, Release Engineer, Platform Engineer, Tester, Verifier, Operations Engineer, Security Auditor, and Coordinator.

## REQUIRED ACTIVITIES

Validate artifact and target; check permission and policy; stage; deploy; monitor; pause or roll back; reconcile configuration and data; record; and maintain the mechanism.

## HUMAN-AI INTERACTION REQUIREMENTS

Show artifact, target, identity, phase, health, evidence, impact, and rollback. Require human approval for production, migration, gate bypass, and destructive actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use progressive rollout, health and business metrics, immutable artifacts, idempotent steps, migration checkpoints, independent control, audit, and rollback.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show deployment phase, target, artifact, health, incidents, evidence freshness, owner, and authorized start, pause, stop, rollback, promote, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Deployment specification; target and identity; rollout and health record; migration and rollback; logs, decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify artifact, target, phase, identity, configuration, method, time, result, impact, and limitation. Execution completion proves neither readiness nor user success.

## VERIFICATION AND ACCEPTANCE TESTS

Verify identity, target, artifact, health, gate, migration, rollback, audit, cleanup, and partial-failure behavior. Test stale health and control-plane loss.

## FAILURE STATES AND ANTI-PATTERNS

No target validation, broad rollout, health blind to user outcome, non-idempotent step, migration without rollback, and deployment success treated as acceptance.

## RECOVERY AND RECONSIDERATION

Pause, roll back or restore, preserve state, communicate, repair data, and reassess impact. Reopen on artifact, target, config, dependency, or health change.

## COMPLETION CRITERIA

Artifact, target, identity, strategy, health, migration, rollback, evidence, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve manifests, logs, artifacts, rollout, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Release, Operations, Platform, Security, Support, and Incident Response.

## RELATED CODEX SECTIONS

267-268 Continuous Delivery and Deployment; 288 Release Management; 295 Environment Promotion; 297 Configuration Delivery; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, deployment, release, security, and operations context, and the integration roadmap. Exact targets, strategies, health, migration, rollback, authority, and acceptance evidence remain unresolved.

# 291

## SECTION 291 - DISTRIBUTION ENGINEERING

CODEX/03_SECTIONS/291_DISTRIBUTION_ENGINEERING.txt

SHA-256 44b85bde1a5c24b85be55152a4d2783ca0ab3d22cafe646807a0503b448a149a

4,864 BYTES / PRESERVED SOURCE IDENTITY

SECTION 291 — Distribution Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Deliver exact artifacts, content, updates, or data to authorized targets with integrity, compatibility, availability, audience control, observability, rollback, and withdrawal.

## REQUIRED OUTCOME

A distribution model with artifact, channel, audience, identity, entitlement, integrity, region, scheduling, throttling, delivery evidence, rollback, withdrawal, and support.

## WHY THIS MATTERS

Distribution can expose the wrong version or data to the wrong audience and can make withdrawal difficult once copied. Delivery success does not prove installation, comprehension, or safe operation.

## APPLICABILITY

Apply when software, firmware, content, data, reports, models, documentation, or configuration is delivered across users, devices, systems, regions, or channels.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no artifact, audience, channel, entitlement, region, or delivery behavior changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use artifact, release, license, identity, authorization, compatibility, CDN, update, privacy, security, support, and rollback baselines.

## REQUIRED INPUTS

Artifact, digest, audience, entitlement, channel, region, version, schedule, integrity, compatibility, telemetry, rollback, withdrawal, and approval.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve audience, channel, entitlement, release, regional, legal, communication, withdrawal, and support decisions. Humans accept delivery and copy risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft distribution plans, segment authorized targets, analyze delivery, and identify anomalies. It must not publish, widen audience, expose data, revoke delivery, or claim receipt without authority.

## APPLICABLE EXISTING AGENT ROLES

Distribution Engineer, Release Engineer, Artifact Manager, Security Auditor, Support Reviewer, Network Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Define audience and channel; validate artifact and entitlement; stage; distribute; monitor receipt and integrity; handle failure and retry; withdraw or replace; support users; and record evidence.

## HUMAN-AI INTERACTION REQUIREMENTS

Show artifact, audience, entitlement, channel, region, identity, delivery, evidence, impact, and limitation. Require human approval for public, sensitive, regional, or irreversible distribution.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use signed artifacts, access and entitlement, staged rollout, regional isolation, throttling, cache invalidation, receipt telemetry, revocation, and fallback channel.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show distribution status, audience, channel, version, integrity, failures, evidence freshness, owner, and authorized pause, narrow, retry, withdraw, replace, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Distribution charter; channel and audience model; entitlement and integrity; rollout and receipt; withdrawal and support; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify artifact, audience, target, channel, version, digest, time, method, receipt, failure, actor, and limitation. Download or delivery event proves only transmission.

## VERIFICATION AND ACCEPTANCE TESTS

Verify audience, entitlement, integrity, compatibility, regional, privacy, channel, retry, receipt, withdrawal, cache, rollback, and support behavior.

## FAILURE STATES AND ANTI-PATTERNS

Wrong audience, unsigned artifact, mutable channel, delivery without entitlement, cache retains withdrawn content, retry duplicates, and transmission treated as operation.

## RECOVERY AND RECONSIDERATION

Pause, revoke or withdraw, preserve delivery evidence, notify affected parties, replace or correct, invalidate copies where possible, and reassess downstream impact.

## COMPLETION CRITERIA

Artifact, audience, channel, entitlement, integrity, region, evidence, withdrawal, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve artifacts, channels, audiences, receipts, failures, withdrawals, decisions, evidence, exceptions, and open gaps. Hand off to Release, Packaging, Installation, CDN, Support, Security, and Operations.

## RELATED CODEX SECTIONS

262 Artifacts; 288 Release; 292 Packaging; 293 Installation; 300 Content Delivery Networks; 355-357 Provisioning, Logistics, and Distribution; 391-400 Lifecycle.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, release, distribution, security, support, and lifecycle context, and the integration roadmap. Exact channels, audiences, entitlements, regions, withdrawal, providers, and acceptance authority remain unresolved.

# 292

## SECTION 292 - PACKAGING

CODEX/03_SECTIONS/292_PACKAGING.txt

SHA-256 ab5ba0640d92de60b3ecd075a94728c84e7269c603b78c95e6288c78eda87688

4,829 BYTES / PRESERVED SOURCE IDENTITY

SECTION 292 — Packaging

## STATUS: CONTENT-COMPLETE

## PURPOSE

Assemble software, firmware, data, content, models, configuration, metadata, notices, and dependencies into an identifiable, installable, distributable, and recoverable package.

## REQUIRED OUTCOME

A package specification with contents, manifest, version, digest, provenance, dependency, configuration, license, platform, installation, integrity, and rollback behavior.

## WHY THIS MATTERS

Packaging can omit files, include secrets, resolve wrong dependencies, lose notices, or create ambiguity between source and shipped behavior. A package that installs is not necessarily complete or authorized.

## APPLICABILITY

Apply when an artifact is assembled for installation, deployment, distribution, archival, or consumption.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no package boundary, contents, metadata, or consumer behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use artifact, build, dependency, license, source, security, platform, installation, distribution, and release baselines.

## REQUIRED INPUTS

Contents, manifest, source, dependencies, versions, configuration, notices, platform, installer, digest, signing, owner, and rollback.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve contents, notices, license, platform, package format, signing, distribution, exceptions, and retirement. Humans verify that no secret or unauthorized material is included.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft manifests, inspect authorized package contents and metadata, and identify omissions. It must not include, expose, publish, sign, or approve package material without authority.

## APPLICABLE EXISTING AGENT ROLES

Packaging Engineer, Build Engineer, Artifact Manager, License Analyst, Security Auditor, Tester, Verifier, and Documentation Writer.

## REQUIRED ACTIVITIES

Define package contract; assemble; verify contents and dependencies; scan secrets and licenses; generate metadata; sign; test installation and rollback; distribute; maintain; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show package, contents, manifest, source, digest, platform, findings, evidence, and limitation. Require human review for signing, license, secret, public, and production package.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use deterministic assembly, manifest and digest, provenance, dependency lock, secret exclusion, signature, platform metadata, atomic install, rollback, and safe cleanup.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show package identity, contents, findings, target platform, install state, evidence freshness, owner, and authorized approve, quarantine, publish, withdraw, replace, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Package specification; manifest and provenance; scan and signature; installation and rollback; license and notice; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify package, digest, contents, source, build, dependencies, platform, method, time, result, actor, and limitation. Package filename or archive creation proves neither completeness nor integrity.

## VERIFICATION AND ACCEPTANCE TESTS

Verify contents, manifest, digest, signature, dependency, secret, license, platform, install, upgrade, rollback, uninstall, and recovery. Test missing, extra, corrupt, and incompatible content.

## FAILURE STATES AND ANTI-PATTERNS

Secrets packaged, mutable dependency, missing notice, undocumented file, non-atomic install, archive copied without digest, and install success treated as release acceptance.

## RECOVERY AND RECONSIDERATION

Quarantine package, preserve evidence, withdraw or replace, rotate secrets, notify consumers, and reassess installed copies. Reopen on contents, dependency, platform, license, or installer change.

## COMPLETION CRITERIA

Contents, manifest, source, dependencies, platform, integrity, signing, installation, rollback, owners, evidence, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve package and manifest versions, scans, signatures, installation results, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Build, Artifact, Distribution, Installation, Release, Security, and Support.

## RELATED CODEX SECTIONS

261 Package Management; 262 Artifact Management; 264-268 Build and Delivery; 291 Distribution; 293 Installation; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, build, artifact, licensing, security, and delivery context, and the integration roadmap. Exact package, platforms, contents, signing, installer, providers, and acceptance authority remain unresolved.

# 293

## SECTION 293 - INSTALLATION ENGINEERING

CODEX/03_SECTIONS/293_INSTALLATION_ENGINEERING.txt

SHA-256 22ce7d10c852c6505fd5d3432a3410f530528639afa9af42b71231d3a3e8e105

4,803 BYTES / PRESERVED SOURCE IDENTITY

SECTION 293 — Installation Engineering

## STATUS: CONTENT-COMPLETE

## PURPOSE

Install, upgrade, configure, verify, roll back, uninstall, and recover a package or capability on its target while protecting data, users, service, and evidence.

## REQUIRED OUTCOME

An installation model with prerequisites, target, identity, package, state transition, configuration, migration, permissions, validation, rollback, support, and cleanup.

## WHY THIS MATTERS

Installation crosses the boundary from artifact to live state. Partial installation, incompatible prerequisites, hidden migration, privilege, or cleanup failure can leave a system unusable or insecure.

## APPLICABILITY

Apply when a package, update, service, configuration, firmware, model, or infrastructure component is installed, upgraded, removed, or restored.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no target state, package, configuration, data, or installation behavior changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use packaging, provisioning, configuration, identity, data migration, compatibility, deployment, backup, rollback, support, and operations baselines.

## REQUIRED INPUTS

Package, target, prerequisites, identity, permissions, configuration, data, migration, state, validation, rollback, support, and acceptance.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve target, privilege, migration, downtime, rollback, support, exception, and completion. Humans decide whether data and service can be changed safely.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may draft installation steps, check prerequisites, analyze logs, and suggest recovery. It must not install, grant privilege, alter data, or claim completion without delegated authority.

## APPLICABLE EXISTING AGENT ROLES

Installation Engineer, Release Engineer, Deployment Engineer, Systems Administrator, Tester, Verifier, Security Auditor, Support Reviewer, and Documentation Writer.

## REQUIRED ACTIVITIES

Validate package and target; check prerequisites and authority; back up; install or upgrade; migrate; validate; record; roll back or repair; clean up; support; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show package, target, prerequisites, privilege, state, data, action, evidence, and limitation. Require human approval for production, data, privileged, destructive, and recovery actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use transactional or staged install, prerequisite checks, state checkpoints, least privilege, atomic migration, rollback, idempotence, logs, and safe uninstall.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show target, package, phase, prerequisites, data state, health, evidence freshness, owner, and authorized start, pause, rollback, repair, verify, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Installation specification; target and prerequisite record; execution and state; migration and validation; rollback and support; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify target, package, version, identity, prerequisite, action, state, method, time, result, actor, and limitation. Installer completion proves only its reported steps.

## VERIFICATION AND ACCEPTANCE TESTS

Verify prerequisite, permission, package, data migration, service, configuration, rollback, uninstall, restart, recovery, logging, and support. Test partial and interrupted installation.

## FAILURE STATES AND ANTI-PATTERNS

Install without backup, hidden privilege, partial state ignored, migration irreversible, uninstall leaves access, reboot surprise, and installer success treated as operational readiness.

## RECOVERY AND RECONSIDERATION

Stop, preserve state and logs, roll back or restore, repair data, communicate, and reassess target and users. Reopen on package, target, dependency, data, or prerequisite change.

## COMPLETION CRITERIA

Package, target, prerequisites, permissions, state, data, validation, rollback, support, evidence, owners, limits, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve installation logs, state, package, migrations, failures, decisions, evidence, exceptions, and open gaps. Hand off to Provisioning, Deployment, Operations, Support, Release, and Security.

## RELATED CODEX SECTIONS

292 Packaging; 294 Provisioning; 290 Deployment; 293 Installation; 295 Environment Promotion; 301-330 Operations; 391-400 Lifecycle.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, packaging, deployment, data, and operations context, and the integration roadmap. Exact targets, prerequisites, migration, privilege, rollback, support, and acceptance authority remain unresolved.

# 294

## SECTION 294 - PROVISIONING

CODEX/03_SECTIONS/294_PROVISIONING.txt

SHA-256 38885263bf0ace49401bac409e78249bdedac51b760a088fbd4d184321dc7a7b

4,917 BYTES / PRESERVED SOURCE IDENTITY

SECTION 294 — Provisioning

## STATUS: CONTENT-COMPLETE

## PURPOSE

Create and initialize authorized resources, identities, devices, accounts, environments, data, and configuration with known ownership, security, and lifecycle.

## REQUIRED OUTCOME

A provisioning model with request, identity, eligibility, allocation, initialization, secrets, configuration, inventory, validation, handoff, recovery, and deprovisioning.

## WHY THIS MATTERS

Provisioning establishes authority and state. Unverified automation can create orphaned resources, excessive privilege, wrong tenant or region, exposed secrets, and no path to clean removal.

## APPLICABILITY

Apply when resources, accounts, devices, tenants, environments, services, or data are created or initialized for use.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no resource, identity, allocation, initialization, access, or lifecycle behavior changes and an approved exact baseline applies.

## PREREQUISITES AND DEPENDENCIES

Use IAM, authorization, infrastructure, configuration, environment, secrets, data, policy, billing, security, support, and retirement baselines.

## REQUIRED INPUTS

Request, requester, owner, eligibility, resource, tenant, region, identity, quota, configuration, secret reference, validation, billing, and deprovisioning.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve eligibility, allocation, identity, access, cost, data, exceptions, and retirement. Humans review high-impact or privileged provisioning and retain appeal.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may validate requests, draft provisioning plans, identify missing metadata, and summarize state. It must not create identities or resources, grant access, choose region, expose secrets, or deprovision without authority.

## APPLICABLE EXISTING AGENT ROLES

Provisioning Engineer, IAM Engineer, Infrastructure Engineer, Platform Engineer, Security Auditor, Data Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Validate request and authority; allocate; create; initialize; configure; inject secrets; register; test; notify and hand off; monitor; reconcile; and deprovision or recover.

## HUMAN-AI INTERACTION REQUIREMENTS

Show requester, owner, resource, tenant, region, identity, cost, data, action, evidence, and limitation. Require human approval for privileged, sensitive, billable, regional, and destructive actions.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use idempotence, least privilege, policy, quotas, ownership tags, secret injection, inventory, approval, audit, rollback, and clean deprovisioning.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show request, resource, owner, access, tenant, region, cost, status, evidence freshness, and authorized approve, reject, provision, restrict, recover, and deprovision controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Provisioning request and policy; resource and identity record; initialization and validation; cost and ownership; handoff, deprovisioning, decision, evidence, and continuity records.

## EVIDENCE REQUIREMENTS

Identify request, requester, owner, resource, identity, tenant, region, configuration, method, time, result, and limitation. Resource creation proves neither correct authorization nor readiness.

## VERIFICATION AND ACCEPTANCE TESTS

Verify eligibility, ownership, identity, access, tenant, region, configuration, secret, quota, cost, inventory, validation, handoff, and deprovisioning. Test duplicate and partial request.

## FAILURE STATES AND ANTI-PATTERNS

Orphaned resources, shared identity, wrong tenant, missing owner, secret in output, quota bypass, provisioning without billing, and no clean removal.

## RECOVERY AND RECONSIDERATION

Restrict or deprovision unsafe resource, preserve state, rotate credentials, correct metadata, notify owner, and reassess data and cost. Reopen on request, policy, provider, resource, or owner change.

## COMPLETION CRITERIA

Request, authority, owner, resource, identity, tenant, region, configuration, cost, evidence, handoff, recovery, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve request, resource, identity, configuration, secrets references, cost, handoff, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Installation, Environment, Platform, Security, Billing, Operations, and Retirement.

## RELATED CODEX SECTIONS

200-205 IAM and Protection; 263 Environment; 275-277 Infrastructure and Cloud; 293 Installation; 355 Device Provisioning; 396 Account Closure; 397-400 Decommissioning.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, identity, infrastructure, security, data, and operations context, and the integration roadmap. Exact resources, identity, tenants, regions, cost, approval, deprovisioning, and acceptance authority remain unresolved.

# 295

## SECTION 295 - ENVIRONMENT PROMOTION

CODEX/03_SECTIONS/295_ENVIRONMENT_PROMOTION.txt

SHA-256 283dc0635c6382fdb1101fb74af0083fc75a7ab6293ce158974cd74ec0b5cf29

4,705 BYTES / PRESERVED SOURCE IDENTITY

SECTION 295 — Environment Promotion

## STATUS: CONTENT-COMPLETE

## PURPOSE

Move an identified artifact, configuration, data change, or capability between environments only after declared evidence, policy, ownership, and readiness conditions are met.

## REQUIRED OUTCOME

A promotion model with source and target environment, artifact, configuration, data, gates, approvals, compatibility, migration, health, rollback, and evidence.

## WHY THIS MATTERS

Promotion can hide environment drift and turn test evidence into an unsupported production claim. Data, identity, provider, capacity, and operations can change the meaning of a result.

## APPLICABILITY

Apply when a candidate moves from development, test, staging, field, canary, or recovery into another environment or population.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no environment, artifact, configuration, data, or audience transition occurs and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use artifact, environment, configuration, testing, acceptance, release, change, data migration, identity, operations, and rollback baselines.

## REQUIRED INPUTS

Candidate, source and target, evidence, configuration, data, gates, compatibility, approvals, migration, health, support, and rollback.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve gate interpretation, target, data, migration, risk, exception, timing, and promotion. Humans decide whether evidence transfers honestly between environments.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may compare environment state, assemble evidence, identify gaps, and draft promotion plans. It must not approve, promote, alter gates, or claim parity without human authority.

## APPLICABLE EXISTING AGENT ROLES

Release Engineer, Environment Manager, Deployment Engineer, Tester, Verifier, Architect, Operations Engineer, Security Auditor, and Coordinator.

## REQUIRED ACTIVITIES

Baseline source and target; compare configuration and dependencies; review evidence; validate data and identity; approve; promote; observe; pause or roll back; record; and update support.

## HUMAN-AI INTERACTION REQUIREMENTS

Show candidate, source, target, differences, gates, evidence, impact, authority, and rollback. Require human approval for production, sensitive data, migration, and exception.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use immutable artifacts, environment-specific config, policy gates, staged promotion, synthetic or controlled data, health, rollback, migration checkpoints, and audit.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show source and target, candidate, drift, gates, approvals, health, evidence freshness, owner, and authorized promote, pause, rollback, restrict, and reopen controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Promotion plan; environment comparison; readiness and approvals; migration and health; rollout and rollback; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify source, target, artifact, config, data, test, approval, method, time, result, impact, and limitation. Promotion proves movement, not target acceptance or user success.

## VERIFICATION AND ACCEPTANCE TESTS

Verify artifact, environment, config, data, identity, gates, compatibility, migration, health, rollback, audit, and partial promotion. Test target drift and failed gate.

## FAILURE STATES AND ANTI-PATTERNS

Environment parity assumed, gate bypass, production data copied, target health stale, configuration overwritten, promotion treated as deployment success, and rollback without data plan.

## RECOVERY AND RECONSIDERATION

Pause or roll back, preserve source and target state, restore data, communicate, and reassess evidence transfer and user impact. Reopen on environment, candidate, dependency, data, or gate change.

## COMPLETION CRITERIA

Source, target, candidate, gates, approvals, config, data, health, rollback, evidence, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve comparisons, approvals, promotions, health, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Deployment, Release, Configuration, Operations, Support, and Governance.

## RELATED CODEX SECTIONS

263 Environment; 267-268 Delivery and Deployment; 290 Deployment; 297 Configuration Delivery; 301-330 Operations; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, environment, release, testing, and operations context, and the integration roadmap. Exact environments, gates, data, identity, health, rollback, and acceptance authority remain unresolved.

# 296

## SECTION 296 - FEATURE MANAGEMENT

CODEX/03_SECTIONS/296_FEATURE_MANAGEMENT.txt

SHA-256 fb28dc8648637a0cd37bd37ecf0dd6b9076479f23cfe6b0b376e7c94938d9a5a

4,781 BYTES / PRESERVED SOURCE IDENTITY

SECTION 296 — Feature Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Govern the activation, targeting, experimentation, monitoring, rollback, retirement, and evidence of conditional product, service, policy, or operational behavior.

## REQUIRED OUTCOME

A feature model with owner, purpose, population, rule, dependency, default, state, metrics, safety, access, expiry, rollback, and retirement.

## WHY THIS MATTERS

Flags and experiments are production control surfaces. Stale or conflicting flags, unsafe targeting, hidden defaults, and unowned experiments can create inconsistent behavior and permanent complexity.

## APPLICABILITY

Apply when behavior can be enabled, disabled, varied, targeted, experimented with, or retired independently of a full artifact release.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no conditional behavior, target population, experiment, configuration, or activation state changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use release, configuration, authorization, experimentation, privacy, analytics, safety, support, and operations baselines.

## REQUIRED INPUTS

Feature, purpose, owner, population, rule, default, dependency, data, metric, expiry, approval, rollout, rollback, and retirement.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve purpose, target, data, experiment, activation, kill switch, expiry, exception, and retirement. Humans decide user and safety impact.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized flag state and metrics, draft targeting and test cases, and identify stale or conflicting features. It must not activate, target, infer sensitive attributes, hide cohorts, or retire flags without authority.

## APPLICABLE EXISTING AGENT ROLES

Feature Manager, Product Manager, Release Engineer, Analyst, Experimenter, Privacy Reviewer, Security Auditor, Tester, Verifier, and Documentation Writer.

## REQUIRED ACTIVITIES

Define purpose and owner; design safe default and targeting; review data and privacy; test; activate progressively; monitor; pause or roll back; expire; measure outcome; and remove.

## HUMAN-AI INTERACTION REQUIREMENTS

Show feature, rule, population, data, owner, state, metric, evidence, and limitation. Require human approval for activation, targeting, sensitive cohort, experiment, rollback, and retirement.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use typed flags, safe defaults, evaluation audit, targeting isolation, kill switch, dependency handling, expiry, configuration version, and behavior observability.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show feature state, population, rule, owner, metric, incidents, evidence freshness, expiry, and authorized activate, pause, roll back, restrict, extend, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Feature catalog; targeting and privacy record; test and rollout; metrics and experiment; rollback and retirement; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify feature, version, rule, population, data, environment, state, method, time, result, owner, and limitation. Activation or engagement proves neither benefit nor safety.

## VERIFICATION AND ACCEPTANCE TESTS

Verify default, targeting, authorization, privacy, accessibility, metrics, kill switch, rollback, expiry, conflict, and removal. Test missing context and evaluation service loss.

## FAILURE STATES AND ANTI-PATTERNS

Stale flag, permanent experiment, unsafe default, sensitive targeting, conflicting flags, no owner, kill switch untested, and engagement treated as outcome.

## RECOVERY AND RECONSIDERATION

Pause or disable, preserve state and cohort evidence, communicate if needed, correct targeting, and reassess users and data. Reopen on metric, population, policy, dependency, or feature change.

## COMPLETION CRITERIA

Purpose, owner, state, rule, population, data, metric, expiry, rollback, evidence, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve catalog, states, targeting, experiments, metrics, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Release, Configuration, Analytics, Privacy, Product, Support, and Operations.

## RELATED CODEX SECTIONS

161 Analytics Engineering; 218 Privacy; 267-268 Delivery and Deployment; 297 Configuration Delivery; 369-371 Experimentation, A/B Testing, and Optimization; 388 Version Management.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, feature, experimentation, privacy, analytics, and operations context, and the integration roadmap. Exact flag system, populations, metrics, authority, expiry, and acceptance evidence remain unresolved.

# 297

## SECTION 297 - CONFIGURATION DELIVERY

CODEX/03_SECTIONS/297_CONFIGURATION_DELIVERY.txt

SHA-256 67cb851a08009769460df3b6b9141263b9b2fb182f1b00ee28ac70f33ad52121

4,526 BYTES / PRESERVED SOURCE IDENTITY

SECTION 297 — Configuration Delivery

## STATUS: CONTENT-COMPLETE

## PURPOSE

Deliver approved configuration changes to exact environments and consumers with version, validation, authorization, observability, rollback, and drift control.

## REQUIRED OUTCOME

A configuration-delivery model with source, target, value references, schema, gate, identity, sequence, dependency, health, rollback, audit, and recovery.

## WHY THIS MATTERS

Configuration delivery can change behavior without a binary release. Wrong target, stale value, secret exposure, ordering, or incomplete rollback can create broad and invisible impact.

## APPLICABILITY

Apply when configuration, policy, feature, environment, routing, threshold, or runtime setting is delivered or activated.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no configuration or runtime behavior changes and an approved exact baseline covers the scope.

## PREREQUISITES AND DEPENDENCIES

Use configuration, source, environment, feature, deployment, identity, secrets, testing, operations, and rollback baselines.

## REQUIRED INPUTS

Change, schema, source, target, value reference, identity, dependencies, gate, sequence, validation, health, rollback, and approval.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve values, target, sequence, security, production, emergency, rollback, and retirement. Humans review secret and safety impact.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may compare configs, draft delivery plans, validate schemas, and identify drift. It must not expose values, deliver, activate, change gates, or claim success without authority.

## APPLICABLE EXISTING AGENT ROLES

Configuration Manager, Deployment Engineer, Release Engineer, Platform Engineer, Security Auditor, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Validate source and schema; resolve target and authority; stage; deliver; verify; observe; pause or roll back; reconcile; record; and retire obsolete configuration.

## HUMAN-AI INTERACTION REQUIREMENTS

Show item, target, schema, value reference, identity, impact, evidence, and limitation without revealing secrets. Require human approval for protected and production change.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use versioned source, schema validation, secret references, staged delivery, atomic or ordered change, drift detection, safe default, rollback, and audit.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show config version, target, drift, health, owner, evidence freshness, dependency, and authorized promote, pause, reconcile, rollback, restrict, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Configuration delivery plan; source and target; schema and validation; rollout and health; rollback and drift; decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify config item, version, target, schema, identity, method, time, result, and limitation. Delivery event proves transmission, not correct activation or behavior.

## VERIFICATION AND ACCEPTANCE TESTS

Verify schema, target, permissions, sequence, dependency, secret boundary, activation, health, drift, rollback, and failure. Test invalid, stale, partial, and conflicting values.

## FAILURE STATES AND ANTI-PATTERNS

Manual edit, secret in delivery log, wrong target, schema bypass, config drift, no rollback, activation without health, and config change invisible to release history.

## RECOVERY AND RECONSIDERATION

Pause, restore prior valid configuration, preserve evidence, rotate secrets, communicate, and reassess behavior. Reopen on config, environment, dependency, policy, or feature change.

## COMPLETION CRITERIA

Source, schema, target, identity, sequence, health, rollback, evidence, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve config versions, delivery logs, targets, drift, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Configuration, Feature, Deployment, Platform, Security, and Operations.

## RELATED CODEX SECTIONS

258 Configuration; 290 Deployment; 295 Environment Promotion; 296 Feature Management; 298 Certificate Management; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, configuration, deployment, security, and operations context, and the integration roadmap. Exact configs, targets, schema, secrets, gates, rollback, and acceptance authority remain unresolved.

# 298

## SECTION 298 - CERTIFICATE MANAGEMENT

CODEX/03_SECTIONS/298_CERTIFICATE_MANAGEMENT.txt

SHA-256 ce6e1c48cd31d45e1efe7f84ae8f83649685c4301828907448a589f6ad2c35f5

4,999 BYTES / PRESERVED SOURCE IDENTITY

SECTION 298 — Certificate Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Govern certificates, trust chains, identities, issuance, validation, renewal, revocation, deployment, monitoring, and retirement across their lifecycle.

## REQUIRED OUTCOME

A certificate model with subject, purpose, authority, key reference, validity, issuance, storage, consumer, renewal, revocation, monitoring, evidence, and recovery.

## WHY THIS MATTERS

Expired, misissued, untrusted, mismatched, or exposed certificates can interrupt services or undermine identity. Renewal automation can also create outages or hide a trust change.

## APPLICABILITY

Apply when certificates, public keys, trust chains, TLS or equivalent identities, signing certificates, or certificate authorities secure a service, device, user, or artifact.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no certificate, trust chain, key reference, validation, renewal, or identity behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use cryptography, key, identity, secrets, DNS, network, application, device, release, monitoring, and incident baselines.

## REQUIRED INPUTS

Certificate, subject, purpose, authority, key reference, validity, consumer, chain, policy, issuance, renewal, revocation, monitoring, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve authority, subject, issuance, renewal, revocation, trust, exception, and emergency replacement. Humans protect private key material and decide trust changes.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized certificate metadata, expiry, chain, and usage; draft renewal plans and tests; and identify mismatches. It must not access private keys, issue, install, revoke, or trust certificates without authority.

## APPLICABLE EXISTING AGENT ROLES

Certificate Manager, Cryptography Engineer, Key Manager, Network Engineer, Security Auditor, Platform Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Inventory; classify and assign owner; issue; validate; deploy; monitor expiry and trust; renew; revoke; replace; test consumer; preserve evidence; and retire certificates and authorities.

## HUMAN-AI INTERACTION REQUIREMENTS

Show certificate identifier, subject, authority, consumer, expiry, chain, key reference, evidence, and limitation without exposing private key. Require human approval for issuance, trust, renewal, revocation, and replacement.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use automated discovery, short validity where appropriate, protected key storage, trust separation, renewal overlap, revocation or status, chain validation, and independent recovery.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show certificate health, subject, authority, expiry, consumer, chain, evidence freshness, owner, and authorized renew, replace, revoke, restrict, recover, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Certificate inventory; authority and trust model; issuance and renewal; consumer deployment; monitoring and revocation; incident, decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify certificate, subject, authority, key reference, chain, consumer, version, method, time, result, and limitation. Expiry scan proves only the scanned inventory and time.

## VERIFICATION AND ACCEPTANCE TESTS

Verify subject, chain, hostname or identity, key binding, expiry, renewal, deployment, revocation, trust, alerting, rollback, and recovery. Test clock skew, authority loss, and expired certificate.

## FAILURE STATES AND ANTI-PATTERNS

Private key exposure, untracked certificates, last-minute renewal, trust-all validation, mismatched identity, expired chain, and renewal automation without rollback or alert owner.

## RECOVERY AND RECONSIDERATION

Replace or revoke, preserve evidence, rotate related keys, restore trust, communicate, and reassess consumers and exposure. Reopen on authority, subject, key, DNS, provider, or policy change.

## COMPLETION CRITERIA

Certificates, authorities, subjects, keys, consumers, validity, renewal, revocation, evidence, recovery, owners, limitations, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve metadata and lifecycle records, not private keys, along with deployments, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Keys, Cryptography, DNS, Network, Platform, Security, and Operations.

## RELATED CODEX SECTIONS

203 Cryptography; 204 Key Management; 200-202 IAM and Authentication; 299 Domain and DNS; 284 Network; 301-330 Operations; 380 Security Patching.

CODEX/03_SECTIONS/298_CERTIFICATE_MANAGEMENT.txt

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, cryptography, identity, network, and operations context, and the integration roadmap. Exact authorities, certificate classes, validity, renewal, revocation, providers, and acceptance authority remain unresolved.

# 299

## SECTION 299 - DOMAIN AND DNS MANAGEMENT

CODEX/03_SECTIONS/299_DOMAIN_AND_DNS_MANAGEMENT.txt

SHA-256 a222537af3a0e9a6ab2821feb1adeb2337c211387ea57089d18ce705ce562a7b

4,862 BYTES / PRESERVED SOURCE IDENTITY

SECTION 299 — Domain and DNS Management

## STATUS: CONTENT-COMPLETE

## PURPOSE

Govern domains, names, registrations, delegation, DNS records, ownership, security, routing, change, expiry, recovery, and retirement.

## REQUIRED OUTCOME

A domain and DNS model with names, owners, registrars, authoritative service, records, TTL, validation, certificates, access, monitoring, change, renewal, and recovery.

## WHY THIS MATTERS

Domain and DNS state controls identity and reachability. Expiry, hijack, stale records, weak delegation, misrouting, and uncontrolled automation can redirect users or take services offline.

## APPLICABILITY

Apply when domains, DNS, names, delegation, service discovery, certificate validation, or routing names affect a system, user, or service.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no domain, name, record, delegation, routing, certificate, or expiry behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use identity, network, certificate, cloud, application, security, configuration, provider, records, and operations baselines.

## REQUIRED INPUTS

Domains, owners, registrars, authorities, records, zones, TTL, providers, access, certificates, services, renewal, monitoring, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve registration, delegation, records, access, provider, renewal, transfer, emergency change, and retirement. Humans verify ownership and high-impact routing.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized DNS metadata, expiry, records, and health; draft changes and tests; and identify anomalies. It must not change delegation, routing, registration, or records without authority.

## APPLICABLE EXISTING AGENT ROLES

DNS Engineer, Network Engineer, Certificate Manager, Cloud Engineer, Security Auditor, Platform Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Inventory domains and zones; verify ownership; secure registrar and DNS access; manage records and TTL; monitor expiry and health; renew; change; recover; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show domain, zone, record, owner, authority, provider, target, evidence, and limitation. Require human approval for delegation, registrar, routing, certificate, and production changes.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use separate registrar and DNS roles, multi-provider or secondary DNS where warranted, DNSSEC where applicable, change audit, health checks, low-risk TTL strategy, and recovery contacts.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show domain health, expiry, authority, records, targets, certificates, incidents, evidence freshness, owner, and authorized update, fail over, restrict, renew, transfer, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

Domain and zone inventory; ownership and access; record and routing model; certificate links; monitoring, renewal, recovery, decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify domain, zone, record, authority, target, provider, owner, method, time, result, and limitation. DNS query response proves only the observed resolver and time.

## VERIFICATION AND ACCEPTANCE TESTS

Verify ownership, delegation, records, TTL, DNSSEC, certificate validation, routing, expiry, provider, monitoring, failover, rollback, and recovery. Test stale, malicious, and unavailable authority.

## FAILURE STATES AND ANTI-PATTERNS

Expired domain, shared registrar account, untracked record, wildcard overreach, stale target, DNS change without audit, resolver cache ignored, and DNS health treated as service health.

## RECOVERY AND RECONSIDERATION

Restrict access, restore known-good records, fail over authority, renew or recover registration, preserve evidence, communicate, and reassess certificate and service impact. Reopen on provider, domain, record, or threat change.

## COMPLETION CRITERIA

Domains, zones, owners, authorities, records, targets, certificates, monitoring, evidence, recovery, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve registration, delegation, records, changes, incidents, certificates, decisions, evidence, exceptions, and open gaps. Hand off to Network, Certificates, Cloud, Security, Platform, Release, and Operations.

## RELATED CODEX SECTIONS

038 Network Architecture; 284 Network Engineering; 298 Certificate Management; 277 Cloud; 300 Content Delivery Networks; 301-330 Operations.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, network, identity, certificate, provider, and operations context, and the integration roadmap. Exact domains, authorities, providers, records, security, renewal, recovery, and acceptance authority remain unresolved.

# 300

## SECTION 300 - CONTENT DELIVERY NETWORKS

CODEX/03_SECTIONS/300_CONTENT_DELIVERY_NETWORKS.txt

SHA-256 d0dc5e39fc6e81b2e10472f06e826f9f771d6720a0a73f88fadf4363619a60ee

4,794 BYTES / PRESERVED SOURCE IDENTITY

SECTION 300 — Content Delivery Networks

## STATUS: CONTENT-COMPLETE

## PURPOSE

Deliver cached or proxied content with controlled origin, identity, cache, integrity, privacy, availability, invalidation, observability, and recovery.

## REQUIRED OUTCOME

A CDN model with content, origin, audience, cache policy, headers, certificates, access, invalidation, regions, provider, metrics, cost, and rollback.

## WHY THIS MATTERS

Caching changes what users receive and for how long. Stale, private, wrong-region, unauthorized, or poisoned content can persist after origin correction and may be hard to withdraw.

## APPLICABILITY

Apply when a CDN, edge cache, reverse proxy, or content acceleration service stores or serves content, APIs, data, or artifacts.

## NON-APPLICABILITY CONDITIONS

Record non-applicability when no cache, origin, content, access, certificate, invalidation, or delivery behavior changes and an approved exact baseline covers it.

## PREREQUISITES AND DEPENDENCIES

Use content, domain, certificate, network, privacy, security, artifact, release, distribution, and operations baselines.

## REQUIRED INPUTS

Content, origin, cache key, TTL, headers, audience, identity, region, certificate, invalidation, provider, metrics, cost, and recovery.

## HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve origin, content, cache, private-data handling, access, provider, invalidation, cost, and emergency purge. Humans accept stale and provider risk.

## AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized CDN metadata, cache behavior, headers, and metrics; draft policy and tests; and identify anomalies. It must not publish, purge, change origin, expose content, or alter cache policy without authority.

## APPLICABLE EXISTING AGENT ROLES

CDN Engineer, Network Engineer, Cloud Engineer, Security Auditor, Release Engineer, Content Engineer, Tester, Verifier, and Operations Engineer.

## REQUIRED ACTIVITIES

Define content and origin; design cache and access; configure certificates and headers; deploy; validate; monitor; invalidate; handle origin or provider failure; control cost; and retire.

## HUMAN-AI INTERACTION REQUIREMENTS

Show content, origin, cache, audience, region, identity, certificate, action, evidence, and limitation. Require human approval for private content, origin, purge, routing, and production change.

## ARCHITECTURE AND SYSTEM IMPLICATIONS

Use correct cache keys, private-content isolation, signed or authenticated requests, origin protection, TTL, purge, versioned URLs, integrity, WAF or rate controls, and fallback.

## OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show origin, cache health, content version, freshness, regions, access, incidents, evidence freshness, cost, owner, and authorized purge, route, restrict, rollback, and retire controls.

## REQUIRED OUTPUTS AND ARTIFACTS

CDN charter; origin and cache model; content and access policy; certificate and header; metrics and purge; incident, decision, evidence, exception, and continuity records.

## EVIDENCE REQUIREMENTS

Identify content, origin, cache key, region, request, policy, method, time, result, actor, and limitation. Cache hit or latency proves neither freshness nor authorization.

## VERIFICATION AND ACCEPTANCE TESTS

Verify cache key, privacy, headers, access, integrity, TTL, invalidation, certificate, origin failover, regions, cost, and recovery. Test stale and poisoned content.

## FAILURE STATES AND ANTI-PATTERNS

Private data cached publicly, stale security content, purge not tested, origin exposed, cache poisoning, wildcard policy, and performance treated as delivery correctness.

## RECOVERY AND RECONSIDERATION

Purge or restrict affected content, protect origin, preserve evidence, restore or replace, rotate credentials, notify consumers, and reassess copies. Reopen on content, origin, provider, policy, or certificate change.

## COMPLETION CRITERIA

Origin, content, cache, access, identity, regions, certificates, freshness, purge, evidence, recovery, owners, and reopening conditions are explicit. Status remains CONTENT-COMPLETE pending review and owner acceptance.

## HANDOFF AND CONTINUITY REQUIREMENTS

Preserve content and cache policy, origins, certificates, purges, incidents, decisions, evidence, exceptions, and open gaps. Hand off to Distribution, DNS, Network, Security, Release, Cloud, and Operations.

## RELATED CODEX SECTIONS

291 Distribution; 298 Certificates; 299 DNS; 284 Network; 277 Cloud; 301-330 Operations; 391-395 Data Retention and Deletion.

## SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, content, network, security, distribution, and operations context, and the integration roadmap. Exact provider, content, origin, cache, privacy, invalidation, regions, cost, and acceptance authority remain unresolved.